Algorithms and Processes of a No-Limit Hold'em Engine

A first-principles reference on poker rules, probability, game theory, engine processes, associated evaluators and solvers, and underlying mathematics

♠ ♡ ♢ ♣

This document specifies the complete mathematical foundations, operational processes, state-machine rules, evaluation algorithms, probability estimators, and game-theoretic decision policies of a No-Limit Texas Hold'em poker engine and its associated solver architectures. Every concept is built from scratch in clear, beginner-friendly language before its formal mathematical derivation and pseudocode specification are presented.

Section 1 introduces the game of poker, the foundational laws of probability, and the core principles of game theory (expected value, Nash equilibrium, bluff-to-value balance, and regret). **Section 2** establishes the complete rules of No-Limit Hold'em, all ten five-card hand rankings with exact combinatorial derivations, pot-size mechanics, pot odds, stack-to-pot ratios, side pots, and essential terminology. **Sections 3 through 9** detail the seven explicit operational processes of the engine and associated architectures: (I) Card Representation and Cryptographic Shuffling, (II) Hand Evaluation across Combinatorial, Bitmask, Prime-Product, Quinary Combinadic, and State-Graph Evaluators, (III) Monte Carlo Equity Rollout, Range Rejection Sampling, and Exact Runout Enumeration, (IV) Betting State-Machine Progression and Short All-In Reopening, (V) Uncalled Bet Refund, Horizontal Side-Pot Slicing, and Showdown Settlement, (VI) Strategic Decision Policy, Nash Push/Fold Equilibrium, and Counterfactual Regret Minimization (CFR), and (VII) Bayesian Opponent Profiling and Hindsight-Free Self-Review. **Section 10** concludes with the seven formal system invariants, multi-layer verification processes, and an end-to-end numerical walkthrough of a complete hand.

Contents

1 Introduction to Poker, Probability, and Game Theory **1**
1.1 What Is a Poker Engine? Overview of the Core Processes . . . 1
1.2 Introduction to Probability from Scratch . . . 1
1.3 Introduction to Game Theory from Scratch . . . 2

2 Rules, Hand Rankings, Pot Size, and Core Terminologies **3**
2.1 The Deck, Positions, Blinds, and Street Progression . . . 3
2.2 Legal Betting Actions and Raising Rules . . . 4
2.3 Complete Five-Card Hand Rankings and Combinatorial Frequencies . . . 4
2.4 Pot Size, Pot Odds, Stack-to-Pot Ratio, and Side Pots . . . 5
2.5 Comprehensive Terminology Reference . . . 6

3 Process I: Card Representation and Cryptographic Shuffling **6**
3.1 The Bijective Integer Encoding Process . . . 6
3.2 The Unbiased Cryptographic Shuffling Process . . . 7

4 Process II: Hand Evaluation and Associated Evaluator Architectures **8**
4.1 The Base-13 Lexicographical Packing Process . . . 8
4.2 The Five-Card Classification Process (EVAL5) . . . 9
4.3 The 21-Subset Seven-Card Evaluation Process (BEST7) . . . 10
4.4 The Single-Pass Bitmask and Lookup-Table Evaluation Process (RANK7FAST) . . . 11
4.5 Processes in Associated Standard Hand Evaluators . . . 13

5 Process III: Equity Estimation, Range Filtering, and Runout Enumeration **14**
5.1 Starting-Hand Isomorphism (1,326 to 169 Classes), Card Removal, and Outs . . . 14
5.2 The Monte Carlo Rollout Process (EQUITY) and Standard Error . . . 15
5.3 The Range-Conditioned Rejection-Sampling Process (EQUITYVSRANGE) . . . 16
5.4 The Exact Turn/River Enumeration and Next-Card Vulnerability Process . . . 17

6 Process IV: The Betting State Machine and Turn-Order Process **18**
6.1 Player Chip Accounting and the Dead-Button Blind Process (STARTHAND) . . . 18
6.2 The Turn-Order and Round-Completion Process (FINDNEXTACTOR) . . . 18
6.3 The Action Application and Cumulative Short All-In Reopening Process . . . 19

7 Process V: Uncalled Bet Refund, Side-Pot Slicing, and Showdown Settlement **20**
7.1 The Uncalled Bet Refund Process (REFUNDUNCALLED) . . . 20
7.2 The Horizontal Side-Pot Slicing Process (COMPUTEPOTS) . . . 21
7.3 The Showdown Evaluation and Odd-Chip Settlement Process (AWARDPOTS) . . . 22

8 Process VI: Strategic Decision Policy, Nash Push/Fold, and Solver Processes 23
8.1 Player Archetypes and Difficulty Interpolation 23
8.2 The Preflop Heuristic Valuation Process (CHENSCORE) 23
8.3 The Short-Stack Nash Push/Fold Equilibrium Process (PUSHFOLD) 24
8.4 The Preflop and Postflop Action Selection Processes 25
8.5 Processes in Associated Full-Game GTO Solvers (CFR, Abstraction, and Subgame Solving) 26

9 Process VII: Bayesian Opponent Profiling and Hindsight-Free Self-Review 27
9.1 The Beta-Binomial Updating Process and Two-Sigma Gate 27
9.2 The Hindsight-Free Post-Showdown Self-Review Process (REVIEWHAND) 28

10 System Invariants, Process Complexity, and Worked Walkthrough 29
10.1 The Seven Formal System Invariants 29
10.2 Summary of Process Complexity and Runtime Benchmarks 30
10.3 Complete End-to-End Numerical Walkthrough of a Hand Across All Seven Processes 30

1 Introduction to Poker, Probability, and Game Theory

1.1 What Is a Poker Engine? Overview of the Core Processes

A poker engine is a deterministic system of mathematical and logical processes that governs a card game from the moment a deck is shuffled to the moment the final chip is awarded. Unlike chess or Go—where both players see the entire board and no randomness occurs—poker is a **stochastic game of imperfect information**. Players make decisions under two simultaneous sources of uncertainty:

1. **Aleatoric uncertainty (chance):** Which community cards will fall from the remaining deck on future betting rounds?
2. **Epistemic uncertainty (hidden information):** Which two private cards are each opponent holding, and what do their bets reveal about those cards?

To manage both rules enforcement and strategic decision-making without ambiguity, a complete No-Limit Hold'em engine separates its work into **seven distinct, sequential processes**:

- **Process I — Card Encoding and Cryptographic Shuffling:** Maps the 52 physical cards to compact integers $0 \ldots 51$ and generates an unbiased permutation of the deck using rejection-sampled cryptographic randomness.
- **Process II — Hand Evaluation and Ranking:** Maps any 5-, 6-, or 7-card collection into a single strictly ordered integer score so that comparing any two hands reduces to a single integer comparison ($V(A) > V(B)$).
- **Process III — Probability, Equity, and Runout Simulation:** Computes a player's expected share of the pot (*equity*) against uniform random hands or narrowed opponent ranges using either Monte Carlo simulation or exact combinatorial enumeration.
- **Process IV — Betting State-Machine Progression:** Advances the dealer button and forced blinds, determines whose turn it is to act, enforces minimum bet/raise increments, and resolves whether a short all-in wager reopens betting for prior actors.
- **Process V — Pot Decomposition and Showdown Settlement:** Refunds uncalled overbets, slices unequal multi-way all-in commitments into horizontal main and side pots, evaluates eligible hands at showdown, and distributes indivisible odd chips.
- **Process VI — Strategic Decision Policy and Equilibrium Solving:** Combines preflop heuristic scoring, precomputed heads-up Nash push/fold equilibrium tables, action-conditioned postflop range floors, bet-pressure discounting, and game-theoretic regret minimization.
- **Process VII — Bayesian Opponent Profiling and Hindsight-Free Self-Review:** Tracks opponent betting frequencies using Beta-Binomial conjugate priors gated by statistical significance, and updates post-decision regret buckets after showdown without peeking at unrevealed (mucked) cards.

1.2 Introduction to Probability from Scratch

Every calculation inside a poker engine rests on five foundational concepts from probability theory:

Definition 1.1 (Sample Space, Events, and Uniform Measure)**.** A **sample space** Ω is the set of all possible elementary outcomes of a random process. In a fair deal from a finite deck, every outcome $\omega \in \Omega$ is equally likely, so the probability of any event $E \subseteq \Omega$ is simply the ratio of favourable outcomes to total possible outcomes:

$$\mathbb{P}(E) = \frac{|E|}{|\Omega|}. \tag{1}$$

Definition 1.2 (Combinations and the Binomial Coefficient)**.** Because the order in which cards are dealt into a hand or onto a board does not affect the resulting poker hand, we count unordered subsets. The number of ways to choose k distinct cards from a pool of n remaining cards is given by the **binomial coefficient** (read "n choose k"):

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!} = \frac{n(n-1)\cdots(n-k+1)}{k(k-1)\cdots 1}. \tag{2}$$

For example, the number of distinct 2-card starting hands from a 52-card deck is $\binom{52}{2} = \frac{52\times 51}{2} = 1,326$, and the number of distinct 5-card hands is $\binom{52}{5} = \frac{52\times 51\times 50\times 49\times 48}{5\times 4\times 3\times 2\times 1} = 2,598,960$.

Definition 1.3 (Conditional Probability and Card Removal)**.** Whenever cards are revealed (such as a player's own 2 hole cards and b community cards on the board), the sample space shrinks from 52 cards to the $52-2-b$ unseen cards. The **conditional probability** of event A given observed information B is:

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}. \tag{3}$$

In poker, conditioning on known cards is called **card removal** (or *blocker effects*), and conditioning on an opponent's betting actions (which make strong hands more likely and weak hands less likely) is called **Bayesian range narrowing**.

Definition 1.4 (Random Variables and Expected Value (EV))**.** A **random variable** X assigns a numerical payoff (measured in chips or big blinds) to each possible outcome of a hand. Its **expected value** $\mathbb{E}[X]$ (or EV) is the probability-weighted average of all possible payoffs:

$$\mathrm{EV} = \mathbb{E}[X] = \sum_{\omega\in\Omega} \mathbb{P}(\omega)\, X(\omega). \tag{4}$$

If a decision has $\mathrm{EV} > 0$ (denoted $+$EV), repeating that decision over thousands of hands yields a long-run profit; if $\mathrm{EV} < 0$ ($-$EV), it yields a long-run loss, regardless of whether any single hand happens to win or lose.

1.3 Introduction to Game Theory from Scratch

While probability tells a player how likely a hand is to improve against a fixed set of cards, **game theory** studies how rational decision-makers interact when each player's payoff depends on the strategies chosen by others.

Plain-English Intuition: Why Game Theory Is Necessary in Poker. Suppose an engine only ever bets when it holds an unbeatable hand and always checks when it holds a weak hand. A observant opponent will simply fold every time the engine bets and win the pot every time the engine checks. Conversely, if the engine bluffs on every hand, the opponent will call every bet and win huge pots. Game theory calculates the exact mathematical mix of **value bets** (betting strong hands for profit), **bluffs** (betting weak hands to force folds), and **defensive calls** so that an opponent cannot profit no matter how they adjust.

Three foundational principles of game theory govern modern poker engines and solvers:

1. **Pure vs. Mixed Strategies:** A *pure strategy* always takes the exact same action in a given situation (e.g., always raising with A♠ A♡). A *mixed strategy* $\sigma(I)$ assigns a probability distribution over legal actions at an information state I (e.g., raising 70% of the time and calling 30% of the time) so that opponents cannot deduce the exact hand from the action.

2. **Nash Equilibrium (Game-Theoretic Optimal / GTO Play):** In a two-player zero-sum game (where every chip won by Player 1 is lost by Player 2), a strategy pair $(\sigma_1^\star, \sigma_2^\star)$ is a **Nash equilibrium** if neither player can increase their expected value by unilaterally changing their strategy:
$$u_1(\sigma_1^\star, \sigma_2^\star) \geq u_1(\sigma_1, \sigma_2^\star) \quad \forall \sigma_1, \qquad u_2(\sigma_1^\star, \sigma_2^\star) \geq u_2(\sigma_1^\star, \sigma_2) \quad \forall \sigma_2. \tag{5}$$
Playing a Nash equilibrium guarantees that the engine is **unexploitable**: even an omniscient opponent who knows the engine's exact strategy probabilities cannot achieve positive expected value against it over the long run (once seat positions are alternated).

3. **Minimum Defense Frequency (MDF) and Bluff-to-Value Ratio:** Suppose an aggressor bets B chips into a pot of P chips on the river. If the aggressor is bluffing, they risk B chips to win P chips. For a bluff to break even (EV $= 0$), the defender must fold with probability $f^\star = \frac{B}{P+B}$. Therefore, to prevent the aggressor from profiting with any two random cards as a bluff, the defender must call or raise with at least the **Minimum Defense Frequency**:
$$\text{MDF} = 1 - f^\star = 1 - \frac{B}{P+B} = \frac{P}{P+B}. \tag{6}$$
Conversely, the defender facing a bet of B into a pot of P must call B to contest a total pot of $P + 2B$, receiving pot odds of $\frac{B}{P+2B}$. To make the defender indifferent between calling and folding with a bluff-catcher, the aggressor's betting range should contain bluffs and value hands in the exact ratio Bluffs : Value $= B : (P + B)$, so the optimal fraction of bluffs inside a polarized river bet is:
$$\alpha^\star_{\text{bluff}} = \frac{B}{P+2B}. \tag{7}$$
For a pot-sized bet ($B = P$), the defender must defend MDF $= \frac{1}{2} = 50\%$ of their range, and the aggressor should balance every 2 value bets with 1 bluff ($\alpha^\star_{\text{bluff}} = \frac{1}{3} \approx 33.3\%$).

2 Rules, Hand Rankings, Pot Size, and Core Terminologies

2.1 The Deck, Positions, Blinds, and Street Progression

Standard No-Limit Texas Hold'em is played with a 52-card French deck comprising 13 **ranks** in each of 4 **suits** (♠ Spades, ♡ Hearts, ♢ Diamonds, ♣ Clubs). Ranks are ordered from lowest to highest as:

$$2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10\ (\text{T}) < \text{J}\ (\text{Jack}) < \text{Q}\ (\text{Queen}) < \text{K}\ (\text{King}) < \text{A}\ (\text{Ace}),$$

where the Ace (A) can also serve as the lowest card (1) to complete a five-high straight (A–2–3–4–5, called the **wheel**). Suits have no relative ranking in Hold'em: A♠ K♠ Q♠ J♠ T♠ and A♡ K♡ Q♡ J♡ T♡ tie and split the pot.

Before each hand begins, one seat is marked with the **dealer button (BTN)**, which rotates one seat clockwise after every hand. To ensure there is always money in the pot to contest, the two players immediately clockwise from the button post forced bets before any cards are dealt:

- **Small Blind (SB):** Posted by the player one seat clockwise from the button, normally equal to half of the big blind ($\texttt{sb} = \lfloor \texttt{bb}/2 \rfloor$).
- **Big Blind (BB):** Posted by the player two seats clockwise from the button, equal to the full minimum bet unit (bb) of the table.
- **Heads-Up Exception (Exactly 2 Active Players):** When only two players remain at the table, standard rules require the button to post the small blind and act *first* preflop, while the other player posts the big blind and acts *first* on all postflop streets (flop, turn, and river).

A single hand progresses through up to four betting rounds (**streets**), followed by a **showdown** if more than one player remains un-folded:

1. **Preflop (0 board cards):** Each active player receives 2 private face-down **hole cards**. Betting begins with the player immediately clockwise from the big blind (called **Under the Gun, UTG**) and proceeds clockwise around the table.
2. **Flop (3 board cards):** One top card is discarded face down (**burn card**) and 3 shared community cards are dealt face up to the centre of the table (**the board**). Betting begins with the first active player clockwise from the button.
3. **Turn (4 board cards):** A second burn card is discarded and a 4th community card is dealt face up, followed by a third round of postflop betting.
4. **River (5 board cards):** A third burn card is discarded and a 5th and final community card is dealt face up, followed by the final round of betting.
5. **Showdown:** All remaining un-folded players reveal their hole cards. Each player forms the single highest-ranking 5-card poker hand from the 7 cards available to them (their 2 hole cards plus the 5 community cards).

2.2 Legal Betting Actions and Raising Rules

When the action reaches a player on any street, let `currentBet` denote the highest street bet placed so far on that street, let $p.\texttt{bet}$ denote the amount that player p has already put in on the current street, and let $\texttt{toCall} = \max(0, \texttt{currentBet} - p.\texttt{bet})$ denote the additional chips required to stay in the hand. Depending on `toCall` and the player's remaining chip `stack`, up to five actions exist:

- **Fold:** Forfeit the hand immediately. The player's previously committed chips remain in the pot, and the player cannot win any portion of the pot.
- **Check:** Pass the action without wagering additional chips. Legal *if and only if* $\texttt{toCall} = 0$.
- **Call:** Commit $\min(p.\texttt{stack}, \texttt{toCall})$ chips to match the current street bet. If $p.\texttt{stack} \leq \texttt{toCall}$, the call is an **all-in call** for the player's entire remaining stack.
- **Bet / Raise:** Increase the street's wager to a target total $\tau > \texttt{currentBet}$ (called a *bet* when $\texttt{currentBet} = 0$, and a *raise* when $\texttt{currentBet} > 0$). In No-Limit Hold'em, a full raise must increase `currentBet` by at least the size of the previous largest full bet or raise increment on that street (`lastRaise`, initialized to `bb`):

$$\tau_{\min} = \max\big(\texttt{currentBet} + \texttt{lastRaise}, \texttt{bb}\big), \qquad \tau_{\max} = p.\texttt{bet} + p.\texttt{stack}. \tag{8}$$

- **All-In:** Wager the entirety of one's remaining stack ($\tau = p.\texttt{bet} + p.\texttt{stack}$). If an all-in wager exceeds `currentBet` by less than `lastRaise` ($\tau - \texttt{currentBet} < \texttt{lastRaise}$), it is a **short all-in**: it is always legal for the all-in player, but it does *not* by itself reopen the right to re-raise for players who have already acted since the last full raise.

2.3 Complete Five-Card Hand Rankings and Combinatorial Frequencies

At showdown, every five-card poker hand falls into one of ten mutually exclusive **categories** $\kappa \in \{0, 1, \ldots, 9\}$. Within any single category, ties are broken by comparing an ordered tuple of up to five **tie-breaker ranks** (the primary pattern ranks followed by the remaining side cards, called **kickers**) from most significant to least significant.

κ	Category Name	Exact Count	Probability	Combinatorial Derivation & Tie-Break Order $(k_0, k_1, \dots)$
9	**Royal Flush**	4	0.000154%	$\binom{4}{1} = 4$ Ace-high straight flushes (A–K–Q–J–T of one suit). Tie-breaker: (12).
8	**Straight Flush**	36	0.001385%	$9 \times \binom{4}{1} = 36$ non-royal 5-card sequences in one suit (5-high wheel through K-high). Tie-breaker: (s), where s is the top rank.
7	**Four of a Kind**	624	0.024010%	$\binom{13}{1}\binom{4}{4}\binom{12}{1}\binom{4}{1} = 13 \times 48 = 624$. Four cards of one rank q plus one kicker k. Tie-breaker: (q, k).
6	**Full House**	3,744	0.144058%	$\binom{13}{1}\binom{4}{3}\binom{12}{1}\binom{4}{2} = 13 \times 4 \times 12 \times 6 = 3,744$. Three of a kind t plus a pair p. Tie-breaker: (t, p).
5	**Flush**	5,108	0.196540%	$\binom{4}{1}\binom{13}{5} - 40 = 4(1,287) - 40 = 5,108$. Any 5 cards of the same suit, not in sequence. Tie-breaker: $(r_0, r_1, r_2, r_3, r_4)$.
4	**Straight**	10,200	0.392465%	$10 \times (4^5 - 4) = 10 \times 1,020 = 10,200$. Five consecutive ranks across mixed suits. Tie-breaker: (s).
3	**Three of a Kind**	54,912	2.112845%	$\binom{13}{1}\binom{4}{3}\binom{12}{2}\binom{4}{1}^2 = 13 \times 4 \times 66 \times 16 = 54,912$. Trips t plus two kickers $k_1 > k_2$. Tie-breaker: (t, k_1, k_2).
2	**Two Pair**	123,552	4.753902%	$\binom{13}{2}\binom{4}{2}^2\binom{11}{1}\binom{4}{1} = 78 \times 36 \times 44 = 123,552$. High pair p_1, low pair p_2, kicker k. Tie-breaker: (p_1, p_2, k).
1	**One Pair**	1,098,240	42.256903%	$\binom{13}{1}\binom{4}{2}\binom{12}{3}\binom{4}{1}^3 = 13 \times 6 \times 220 \times 64 = 1,098,240$. Pair p plus kickers $k_1 > k_2 > k_3$. Tie-breaker: (p, k_1, k_2, k_3).
0	**High Card**	1,302,540	50.117739%	$\left(\binom{13}{5} - 10\right)(4^5 - 4) = 1,277 \times 1,020 = 1,302,540$. Five unpaired, non-sequential, non-suited ranks: $(r_0, r_1, r_2, r_3, r_4)$.
	Total Hands	**2,598,960**	**100.000000%**	$\binom{52}{5} = 2,598,960$ distinct 5-card subsets (7,462 equivalence classes).

Table 1: Complete combinatorial breakdown of all $\binom{52}{5} = 2,598,960$ five-card hands in Texas Hold'em.

Remark 2.1 (Note on Royal Flush vs. Straight Flush Categorisation). In mathematical evaluation algorithms, a Royal Flush (A–K–Q–J–T suited) is naturally the highest-ranking Straight Flush ($\kappa = 8$ with top rank $s = 12$). Separating it as $\kappa = 9$ for human display or keeping it as $\kappa = 8, s = 12$ inside the evaluator produces the exact same strict ordering over all 2,598,960 hands.

2.4 Pot Size, Pot Odds, Stack-to-Pot Ratio, and Side Pots

Understanding how chip commitments translate into mathematical incentives requires four quantitative definitions:

1. **Total Pot Size (P):** At any decision point, the total pot P is the sum of all chips committed across prior and current streets by all players (including players who have since folded). If a player faces a call cost of $C = \texttt{toCall}$ into a pot of size P (where P already includes the opponent's bet), calling C contests a final pot of $P + C$.

2. **Pot Odds and Break-Even Equity ($q^\star$):** Suppose a player calls C chips into a current pot P, and no further betting occurs on later streets. If the player's probability of winning (or expected pot share) is $q \in [0, 1]$, the expected value of calling is:

$$\mathrm{EV}(\textsc{Call}) = q(P + C) - C = qP - (1 - q)C. \tag{9}$$

Since folding always yields $\mathrm{EV}(\textsc{Fold}) = 0$, calling is profitable ($\mathrm{EV}(\textsc{Call}) \geq 0$) if and only if the player's equity q meets or exceeds the **pot odds threshold** $q^\star$:

$$q \geq q^\star = \texttt{potOdds} = \frac{C}{P + C}. \tag{10}$$

For example, if the pot before an opponent's bet is 100 and the opponent bets 50, the pot is now $P = 150$ and the cost to call is $C = 50$. The pot odds are $\frac{50}{150+50} = \frac{50}{200} = 25.0\%$: winning more than 1 time in 4 makes the call profitable.

3. **Effective Stack (S_{eff}) and Stack-to-Pot Ratio (SPR):** A player with 5,000 chips cannot win 5,000 chips from an opponent who only has 400 chips. In a heads-up confrontation between players A and B, the **effective stack** is $S_{\text{eff}} = \min(\texttt{stack}_A, \texttt{stack}_B)$. At the start of the flop, the **Stack-to-Pot Ratio** is defined as:

$$\mathrm{SPR} = \frac{S_{\text{eff}}}{P_{\text{flop}}}. \tag{11}$$

Low SPR ($\mathrm{SPR} \leq 3$) means a single pot-sized bet and call commits the entire remaining stack, favouring high pairs (A A, K K, Q Q) and top pair; high SPR ($\mathrm{SPR} \geq 10$) allows three full streets of geometric betting, favouring suited connectors and small pairs that can make disguised straights, flushes, or full houses (**implied odds**).

4. **Main Pot and Side Pots:** When one or more players go all-in with smaller stacks than remaining active players, the engine slices the total pot horizontally by commitment depth. The **main pot** is the lowest slice contested by all non-folded players; each subsequent **side pot** contains the excess chips matched only by the deeper-stacked players and can only be won by those deeper-stacked players.

2.5 Comprehensive Terminology Reference

Table 2 summarises the core operational, probabilistic, and game-theoretic terms used throughout the seven engine processes.

3 Process I: Card Representation and Cryptographic Shuffling

3.1 The Bijective Integer Encoding Process

The first process in any high-speed poker engine is representing the 52 cards of the deck so that extracting a card's rank or suit requires zero memory allocations and zero string comparisons. Every card is encoded as a single integer $c \in \{0, 1, \ldots, 51\}$ via the bijection:

$$c = 4\rho + \sigma, \qquad \rho(c) = \lfloor c/4 \rfloor = c \gg 2, \qquad \sigma(c) = c \bmod 4 = c \,\&\, 3, \tag{12}$$

where $\rho \in \{0, 1, \ldots, 12\}$ is the zero-indexed rank ($0 \mapsto 2, 1 \mapsto 3, \ldots, 8 \mapsto \mathrm{T}, 9 \mapsto \mathrm{J}, 10 \mapsto \mathrm{Q}, 11 \mapsto \mathrm{K}, 12 \mapsto \mathrm{A}$) and $\sigma \in \{0, 1, 2, 3\}$ is the suit index ($0 \mapsto \spadesuit, 1 \mapsto \heartsuit, 2 \mapsto \diamondsuit, 3 \mapsto \clubsuit$).

Plain-English Intuition: Why Bitwise Shifts Replace Division. In binary (base 2), multiplying an integer ρ by 4 shifts its bits two places to the left ($\rho \ll 2$), leaving the lowest two bits empty to store the suit $\sigma \in \{0, 1, 2, 3\}$ (which in binary is $00_2, 01_2, 10_2, 11_2$). To recover the rank ρ, shifting right by two bits ($c \gg 2$) drops the suit bits in a single CPU cycle. To recover the suit σ, bitwise AND with 3 ($c \,\&\, 3$, where $3 = 11_2$) masks out everything except the lowest two bits. Furthermore, $1 \ll \rho$ creates a 13-bit rank flag where bit ρ is set to 1, allowing an entire 7-card hand's ranks to be combined via bitwise OR.

Term	Domain	First-Principles Definition
Hole Cards (H)	Rules	The two private face-down cards dealt to each player preflop.
Board (B)	Rules	Up to five shared face-up community cards dealt across the flop (3), turn (4), and river (5).
Kicker	Ranking	An unpaired side card used to break ties between hands of the same category and primary rank (e.g., A♠ K♢ beats A♡ Q♣ on A♣ 8♠ 4♡ 2♢ 3♣ via the King kicker).
Outs	Probability	Unseen cards remaining in the stub that will improve a drawing hand to a likely winning hand on the next street.
Equity (q or e)	Probability	A player's expected fraction of the pot ($q \in [0, 1]$) averaged over all possible remaining board runouts and opponent holdings, counting an s-way tie as $1/s$ of a win.
Range ($\mathcal{R}$)	Game Theory	The probability distribution over all 1,326 two-card holdings that a player could plausibly hold given their actions so far in the hand.
VPIP / PFR	Profiling	*Voluntarily Put Money in Pot* (percentage of hands where a player calls or raises preflop) and *Preflop Raise* (percentage of hands raised preflop).
3-Bet / 4-Bet	Betting	The third or fourth bet in a sequence on a single street (preflop, the blinds count as the 1st bet, an open-raise is the 2nd bet, and a re-raise is a *3-bet*).
Muck	Showdown	Folding or discarding one's hole cards face down without revealing them to the table.
Dead Money	Settlement	Chips in the pot contributed by players who have subsequently folded; dead money increases every remaining player's pot odds.
Counterfactual Regret	Solver Math	How much more expected value a player would have achieved at an information set had they chosen a specific action instead of their current mixed strategy.

Table 2: Foundational poker, probability, and game-theory terminology.

3.2 The Unbiased Cryptographic Shuffling Process

A full 52-card deck has $52! \approx 8.0658 \times 10^{67}$ possible orderings. Since $\log_2(52!) \approx 225.58$ bits, any standard 32-bit or 64-bit pseudo-random number generator (whose internal state is only 64 or 128 bits) can generate only an infinitesimal fraction of all legal decks. Moreover, taking a raw 32-bit random integer $X \in \{0, \ldots, 2^{32} - 1\}$ modulo n ($X \bmod n$) introduces **modulo bias** whenever n does not divide 2^{32}, because the remainder $r = 2^{32} \bmod n$ causes the first r outcomes to have one extra preimage.

The shuffling process eliminates both weaknesses by pairing a hardware-seeded cryptographic random source with **rejection sampling** (Algorithm 1).

Algorithm 1: CRYPTOGRAPHICSHUFFLE — unbiased rejection sampling and in-place Fisher–Yates permutation

Input: deck array $D[0 \dots 51]$; integer bound $n \geq 1$
Output: uniformly permuted deck D over all 52! orderings

Procedure SECURERANDOMINT(n):
  if $n \leq 1$ **then return** 0
  $L \leftarrow \lfloor 2^{32}/n \rfloor \cdot n$ ▷ *largest multiple of n within* 2^{32}
  repeat
    Draw 32-bit cryptographic random integer $X \sim \text{Uniform}(\{0, 1, \dots, 2^{32} - 1\})$
  until $X < L$ ▷ *reject the incomplete tail* $[L, 2^{32})$
  return $X \bmod n$
Procedure SHUFFLEDECKINPLACE(D):
  for $i \leftarrow 51$ **downto** 1 **do**
    $j \leftarrow$ SECURERANDOMINT($i + 1$) ▷ *uniform index in* $\{0, \dots, i\}$
    Swap $D[i] \leftrightarrow D[j]$
  return D

Proposition 3.1 (Exact Uniformity of the Rejection-Sampled Shuffle Process)**.** *For every* $n \in \{1, \dots, 52\}$, SECURERANDOMINT(n) *returns each integer* $k \in \{0, \dots, n-1\}$ *with probability exactly* $1/n$ *in at most* $1 + 52/2^{32} < 1.00000002$ *expected draws, and* SHUFFLEDECKINPLACE *produces each of the* 52! *deck permutations with probability exactly* $1/52!$.

Proof. Because $L = \lfloor 2^{32}/n \rfloor \cdot n$ is an exact multiple of n, the acceptance region $\{0, 1, \dots, L-1\}$ partitions into n residue classes modulo n, each of identical cardinality $M = \lfloor 2^{32}/n \rfloor$. Conditioning on $X < L$, every residue $k \in \{0, \dots, n-1\}$ has probability $M/(nM) = 1/n$. In the Fisher–Yates loop, step i chooses among $i + 1$ remaining elements uniformly and independently for $i = 51, 50, \dots, 1$, generating $\prod_{i=1}^{51}(i+1) = 52!$ distinct execution paths, each with probability $1/52!$. □

4 Process II: Hand Evaluation and Associated Evaluator Architectures

4.1 The Base-13 Lexicographical Packing Process

During a hand evaluation or Monte Carlo simulation, the engine must compare millions of five-card and seven-card hands per second. Comparing two hands via multi-branch conditional logic (first checking the category κ, then checking the first tie-breaker k_0, then k_1, and so on) causes severe CPU branch-prediction penalties.

To eliminate branching during comparisons, the evaluation process packs a hand's category $\kappa \in \{0, \dots, 8\}$ and its tie-breaker tuple $k = (k_0, \dots, k_{m-1}) \in \{0, \dots, 12\}^m$ (with $m \leq 5$, zero-padded on the right to length 5) into a single integer via a **6-digit base-13 positional number** (Algorithm 2):

$$V(\kappa, k) = \kappa \cdot 13^5 + \sum_{i=0}^{4} k_i \cdot 13^{4-i}, \qquad \text{where } k_i = 0 \text{ for } i \geq m, \text{ and } 13^5 = 371,293. \tag{13}$$

Algorithm 2: ENCODE — pack category and tie-breaker ranks into a single base-13 integer

Input: category $\kappa \in \{0, \ldots, 8\}$; tie-breaker tuple $k = (k_0, \ldots, k_{m-1})$ with $m \leq 5$
Output: strictly ordered integer score $V \in [0,\ 3,341,636]$

$v \leftarrow \kappa$
for $i \leftarrow 0$ **to** 4 **do**
  $v \leftarrow 13 \cdot v + (k_i \text{ if } i < m \text{ else } 0)$
return v

Proposition 4.1 (Order Isomorphism of Base-13 Score Packing)**.** *For any two five-card hands H and H' with category/tie-breaker representations (κ, k) and (κ', k'), $V(\kappa, k) > V(\kappa', k')$ if and only if $(\kappa, k) \succ_{\text{lex}} (\kappa', k')$ under the official rules of Texas Hold'em. Moreover, zero-padding tuples of length $m < 5$ never alters comparisons.*

Proof. Let $(d_0, d_1, d_2, d_3, d_4, d_5) = (\kappa, k_0, k_1, k_2, k_3, k_4) \in \{0, \ldots, 12\}^6$. At the first index $j \in \{0, \ldots, 5\}$ where $d_j \neq d'_j$, suppose $d_j > d'_j$, so $d_j - d'_j \geq 1$. The maximum possible countervailing contribution from all less-significant positions $i > j$ is:

$$\sum_{i=j+1}^{5} 12 \cdot 13^{5-i} = 12 \cdot \frac{13^{5-j} - 1}{13 - 1} = 13^{5-j} - 1 < 1 \cdot 13^{5-j}.$$

Thus $V(\kappa, k) - V(\kappa', k') \geq 13^{5-j} - (13^{5-j} - 1) = 1 > 0$. Finally, two hands of the same category κalways share the exact same unpadded tuple length $m(\kappa) \in \{1, 2, 3, 4, 5\}$, so both receive identical trailing zeros in positions $m(\kappa) \ldots 4$; hands of different categories $\kappa \neq \kappa'$ are already separated at $j = 0$ by at least $13^5 = 371,293$. □

4.2 The Five-Card Classification Process (EVAL5)

Given an exact 5-card subset $C = (c_0, c_1, c_2, c_3, c_4)$, the five-card classification process (Algorithm 3) builds a 13-element rank histogram $cnt[0 \ldots 12]$, sorts the distinct present ranks into a descending list U, groups each rank $r \in U$ into a bucket $G[cnt[r]]$ by its multiplicity (1, 2, 3, or 4), and tests for flushes and straights.

Algorithm 3: EVAL5 — classify and score a 5-card hand from rank multiplicities and suit parity

Input: five distinct cards $C = (c_0, c_1, c_2, c_3, c_4)$
Output: base-13 score $V(C) = \text{ENCODE}(\kappa, k)$

1 Extract ranks $R \leftarrow (\rho(c_0), \ldots, \rho(c_4))$ and suits $S \leftarrow (\sigma(c_0), \ldots, \sigma(c_4))$
2 *flush* $\leftarrow (S_0 = S_1 = S_2 = S_3 = S_4)$
3 Build multiplicity histogram $cnt[r] \leftarrow |\{i : R_i = r\}|$ for $r \in \{0, \ldots, 12\}$
4 $U \leftarrow$ distinct ranks with $cnt[r] > 0$, sorted in strictly descending order
5 Partition U by multiplicity into lists $G[1], G[2], G[3], G[4]$ (each inheriting descending order)
6 *straight* $\leftarrow$ **false**; $s \leftarrow -1$
7 **if** $|U| = 5$ **then**
8 | **if** $U_0 - U_4 = 4$ **then** *straight* $\leftarrow$ **true**; $s \leftarrow U_0$ ▷ *standard consecutive straight*
9 | **else if** $U = (12, 3, 2, 1, 0)$ **then** *straight* $\leftarrow$ **true**; $s \leftarrow 3$ ▷ *wheel straight A-2-3-4-5*
10 **if** *straight* $\wedge$ *flush* **then return** $\text{ENCODE}(8, (s))$
11 **if** $|G[4]| = 1$ **then return** $\text{ENCODE}(7, (G[4]_0, G[1]_0))$
12 **if** $|G[3]| = 1 \wedge |G[2]| = 1$ **then return** $\text{ENCODE}(6, (G[3]_0, G[2]_0))$
13 **if** *flush* **then return** $\text{ENCODE}(5, U)$
14 **if** *straight* **then return** $\text{ENCODE}(4, (s))$
15 **if** $|G[3]| = 1$ **then return** $\text{ENCODE}(3, (G[3]_0, G[1]_0, G[1]_1))$
16 **if** $|G[2]| = 2$ **then return** $\text{ENCODE}(2, (G[2]_0, G[2]_1, G[1]_0))$
17 **if** $|G[2]| = 1$ **then return** $\text{ENCODE}(1, (G[2]_0, G[1]_0, G[1]_1, G[1]_2))$
18 **return** $\text{ENCODE}(0, U)$

Proposition 4.2 (Span-Contiguity Straight Detection and Wheel Rank Assignment)**.** *Let $U = (U_0 > U_1 > U_2 > U_3 > U_4)$ be five strictly decreasing integers. Then $U_0 - U_4 = 4$ if and only if $U = (U_0, U_0 - 1, U_0 - 2, U_0 - 3, U_0 - 4)$. Furthermore, assigning $s = 3$ (rank 5) to the Ace-low wheel $U = (12, 3, 2, 1, 0)$ places it strictly below the 6-high straight ($s = 4$) and makes $(12, 3, 2, 1, 0)$ impossible to reach as a High-Card hand ($\kappa = 0$) or ordinary Flush ($\kappa = 5$).*

Proof. Since the U_i are strictly decreasing integers, each consecutive difference satisfies $U_i - U_{i+1} \geq 1$ for $i = 0, 1, 2, 3$. Summing the four differences yields $U_0 - U_4 = \sum_{i=0}^{3}(U_i - U_{i+1}) \geq 4$, with equality if and only if every step equals 1. For the wheel $U = (12, 3, 2, 1, 0)$, Lines 9–10 and 14 route the hand to $\kappa = 8$ (if suited) or $\kappa = 4$ (if unsuited) with $s = 3 < 4$, ensuring 5-high straight $<$ 6-high straight. □

4.3 The 21-Subset Seven-Card Evaluation Process (BEST7)

At showdown, a player holds 7 cards (2 hole cards plus 5 board cards) and forms the best 5-card subset. Because $\binom{7}{5} = \frac{7 \times 6}{2} = 21$, the direct combinatorial process (Algorithm 4) enumerates all 21 five-card subsets, calls EVAL5 on each, and returns both the maximum score $V^\star$ and the winning 5-card subset $C^\star$ (used to highlight the winning five cards at showdown).

Algorithm 4: BEST7 — exhaustive 21-subset 7-card evaluation returning both score and winning 5-card subset

Input: collection $A = (a_0, \ldots, a_{n-1})$ of $n \in \{5, 6, 7\}$ cards
Output: pair $(V^\star, C^\star)$ where $V^\star = \max_{C \subset A, |C|=5} \text{EVAL5}(C)$

$V^\star \leftarrow -1$; $C^\star \leftarrow$ **nil**
for $i_0 \leftarrow 0$ **to** $n-5$ **do**
  for $i_1 \leftarrow i_0 + 1$ **to** $n-4, \ldots, i_4 \leftarrow i_3 + 1$ **to** $n-1$ **do**
    $C \leftarrow (a_{i_0}, a_{i_1}, a_{i_2}, a_{i_3}, a_{i_4})$
    $v \leftarrow \text{EVAL5}(C)$
    if $v > V^\star$ **then** $V^\star \leftarrow v$; $C^\star \leftarrow C$
return $(V^\star, C^\star)$

4.4 The Single-Pass Bitmask and Lookup-Table Evaluation Process (RANK7FAST)

While BEST7 is ideal at showdown (where it runs once per player and identifies the exact 5 cards to highlight), calling EVAL5 21 times per hand inside a Monte Carlo rollout is unnecessary overhead. The fast evaluation process (RANK7FAST, Algorithm 5) computes the exact same base-13 score in a **single pass over the 7 cards** without allocating arrays or enumerating subsets, using three precomputed 8,192-entry lookup tables indexed by any 13-bit rank mask $m \in \{0, 1, \ldots, 2^{13} - 1 = 8,191\}$:

1. `POPC`$[m]$: Population count (number of set 1-bits in m).
2. `STR8`$[m]$: Highest straight top-rank $s \in \{3, 4, \ldots, 12\}$ present in m (testing $(m \,\&\, (31 \ll r)) = (31 \ll r)$ from $r = 8$ down to 0, and checking $(m \,\&\, \texttt{0x100F}) = \texttt{0x100F}$ for the wheel A–2–3–4–5 where $2^{12} + 2^3 + 2^2 + 2^1 + 2^0 = 4111 = \texttt{0x100F}$), or -1 if no straight exists.
3. `TOP5T`$[m]$: Precomputed base-13 sum $\sum_{i=0}^{4} r_i \cdot 13^{4-i}$ of the five highest set bits $r_0 > r_1 > r_2 > r_3 > r_4$ of m.

Algorithm 5: RANK7FAST — single-pass bitmask and lookup-table 7-card evaluator

Input: array A of $n \in \{5, 6, 7\}$ encoded cards
Output: exact base-13 score $V(A)$ identical to BEST7(A)

$s_0, s_1, s_2, s_3 \leftarrow 0, 0, 0, 0$; $all \leftarrow 0$; $cnt[0 \dots 12] \leftarrow 0$
for $i \leftarrow 0$ **to** $n - 1$ **do**
  $c \leftarrow A_i$; $r \leftarrow c \gg 2$; $u \leftarrow c \,\&\, 3$; $b \leftarrow 1 \ll r$
  $all \leftarrow all \mid b$; $cnt[r] \leftarrow cnt[r] + 1$; $s_u \leftarrow s_u \mid b$
$f_m \leftarrow s_0$ if $\texttt{POPC}[s_0] \geq 5$, s_1 if $\texttt{POPC}[s_1] \geq 5$, s_2 if $\texttt{POPC}[s_2] \geq 5$, s_3 if $\texttt{POPC}[s_3] \geq 5$, else 0
if $f_m \neq 0$ **then** ▷ *at most one suit can have* ≥ 5 *of 7 cards*
  $sf \leftarrow \texttt{STR8}[f_m]$
  if $sf \geq 0$ **then return** $8 \cdot 13^5 + sf \cdot 13^4$ ▷ *Straight Flush*
  return $5 \cdot 13^5 + \texttt{TOP5T}[f_m]$ ▷ *Flush*
Scan $r = 12 \dots 0$ to record top quad q_0, up to two trips $t_0 > t_1$, and up to three pairs $p_0 > p_1 > p_2$
if $q_0 \geq 0$ **then return** $7 \cdot 13^5 + q_0 \cdot 13^4 + (\max_{r \neq q_0, cnt[r] > 0} r) \cdot 13^3$ ▷ *Four of a Kind*
if $t_0 \geq 0 \wedge (t_1 \geq 0 \vee p_0 \geq 0)$ **then return** $6 \cdot 13^5 + t_0 \cdot 13^4 + \max(t_1, p_0) \cdot 13^3$ ▷ *Full House*
$st \leftarrow \texttt{STR8}[all]$
if $st \geq 0$ **then return** $4 \cdot 13^5 + st \cdot 13^4$ ▷ *Straight*
if $t_0 \geq 0$ **then return** $3 \cdot 13^5 + t_0 \cdot 13^4 + k_0 \cdot 13^3 + k_1 \cdot 13^2$ (top 2 kickers $k_0 > k_1$ with $cnt[k] = 1$)
if $p_1 \geq 0$ **then return** $2 \cdot 13^5 + p_0 \cdot 13^4 + p_1 \cdot 13^3 + (\max_{r \notin \{p_0, p_1\}, cnt[r] > 0} r) \cdot 13^2$ ▷ *Two Pair*
if $p_0 \geq 0$ **then return** $1 \cdot 13^5 + p_0 \cdot 13^4 + k_0 \cdot 13^3 + k_1 \cdot 13^2 + k_2 \cdot 13$ (top 3 kickers $k_0 > k_1 > k_2$)
return $\texttt{TOP5T}[all]$ ▷ *High Card* ($\kappa = 0$)

Proposition 4.3 (Pigeonhole Suit Uniqueness and Early Flush Return in RANK7FAST)**.** *In any 7-card hand A ($|A| \leq 7$), at most one suit $u \in \{0, 1, 2, 3\}$ can satisfy* $\texttt{POPC}[s_u] \geq 5$*, and whenever such a suit exists, no Four-of-a-Kind ($\kappa = 7$) or Full House ($\kappa = 6$) can simultaneously exist in A.*

Proof. First, if two distinct suits each contained at least 5 cards, the hand would require at least $5 + 5 = 10 > 7$ cards, contradicting $|A| \leq 7$. Second, suppose suit u has $\texttt{POPC}[s_u] \geq 5$ cards. Since all cards in s_u share suit u, their ranks must be 5 distinct ranks r_1, r_2, r_3, r_4, r_5. At most $7 - 5 = 2$ cards remain in $A \setminus s_u$. Even if both remaining cards pair ranks in s_u, the maximum multiplicity profile across all 13 ranks is either $(3, 1, 1, 1, 1)$ (Three of a Kind) or $(2, 2, 1, 1, 1)$ (Two Pair); neither $(4, 1, \dots)$ nor $(3, 2, \dots)$ can be formed with only 2 extra cards. Hence checking $f_m \neq 0$ before checking for Four-of-a-Kind or Full House is mathematically exact. □

To convert a packed integer score $v = V(\kappa, k)$ back into a human-readable category and rank description, the inverse process (DECODE, Algorithm 6) extracts the base-13 digits from least significant to most significant.

Algorithm 6: DECODE — unpack a base-13 score into its category and tie-breaker ranks

Input: packed score $v \in [0,\ 3,341,636]$
Output: category $\kappa \in \{0, \dots, 8\}$ and tie-breaker array $k = (k_0, k_1, k_2, k_3, k_4)$

$x \leftarrow v$
for $i \leftarrow 4$ **downto** 0 **do**
  $k_i \leftarrow x \bmod 13$; $x \leftarrow \lfloor x / 13 \rfloor$
$\kappa \leftarrow x$
return (κ, k)

4.5 Processes in Associated Standard Hand Evaluators

While RANK7FAST achieves **1.77 million** 7-card evaluations per second using only **48 KB** of lookup tables ($3 \times 8,192$ 16/32-bit integers, fitting inside L1 CPU cache), standalone poker evaluators trade larger memory footprints for even fewer instructions per hand. Three standard evaluator processes dominate the literature (Table 3):

1. **The Prime-Product Perfect-Hash Process (Cactus Kev / Paul Senzee):** Assigns the first 13 prime numbers to the 13 ranks:

$$(2, 3, 4, 5, 6, 7, 8, 9, \mathrm{T}, \mathrm{J}, \mathrm{Q}, \mathrm{K}, \mathrm{A}) \mapsto (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41).$$

   Each card is encoded as a 32-bit word packing its prime p_r, rank nibble r, suit bit 2^σ, and rank bit 2^{16+r}. For any 5-card hand, if all 5 cards share a suit bit ($(c_1 \,\&\, c_2 \,\&\, c_3 \,\&\, c_4 \,\&\, c_5 \,\&\, \texttt{0xF000}) \neq 0$), a single 8,192-entry lookup on the bitwise OR of the rank bits returns the Flush or Straight-Flush rank. Otherwise, if all 5 rank bits are distinct ($\texttt{unique5}[\mathit{rankOr}] \neq 0$), it returns the Straight or High-Card rank immediately. For paired hands, the Fundamental Theorem of Arithmetic guarantees that the product of the five primes $P = \prod_{i=1}^{5} p_{r_i}$ uniquely identifies the rank multiset among the 4,888 possible paired 5-card rank multisets, which Paul Senzee's perfect hash maps in $O(1)$ without binary search.

2. **The Quinary Combinadic Dynamic-Programming Process (`PHEvaluator` / `@pokertools`):** Used by HenryRLee's `PokerHandEvaluator` (`PHEvaluator`) and `@pokertools/evaluator` to evaluate 7-card hands directly without iterating over 21 subsets. A 7-card hand's rank histogram is a 13-tuple $q = (q_0, q_1, \ldots, q_{12}) \in \{0, 1, 2, 3, 4\}^{13}$ satisfying $\sum_{r=0}^{12} q_r = 7$ (a 13-digit base-5, or **quinary**, number). How many valid 13-tuples of $\{0, 1, 2, 3, 4\}^{13}$ sum to 7? By the generating function coefficient $[x^7](1 + x + x^2 + x^3 + x^4)^{13} = [x^7](1 - x^5)^{13}(1 - x)^{-13}$:

$$N_{\text{quinary}}(13, 7) = \binom{19}{7} - 13\binom{14}{2} = 50,388 - 1,183 = \mathbf{49,205}. \tag{14}$$

   Using a precomputed dynamic-programming table $\mathrm{DP}[\ell][r][k]$ (the number of ways to distribute k remaining cards across ranks $\geq r$ when the current rank has multiplicity ℓ), the quinary vector q is mapped bijectively into $\{0, \ldots, 49,204\}$ via:

$$h(q) = \sum_{r=0}^{12} \mathrm{DP}[q_r][r] \left[7 - \sum_{j=0}^{r-1} q_j \right]. \tag{15}$$

   Looking up $\texttt{NO_FLUSH_7}[h(q)]$ returns the exact equivalence class in 1..7,462 in $O(1)$ using only $\approx$ 2.5–5 **MB** of tables, achieving **10M–12M evals/s** in TypeScript/JavaScript and **100M–115M evals/s** in C++/C#.

3. **The 7-Step Directed Acyclic State-Graph Process (`TwoPlusTwo` / `HandRanks.dat`):** Compiles the entire 7-card evaluation process into a deterministic finite automaton stored as a flat array `HR` of 32,487,834 32-bit integers (**128 MB**). Given 7 cards $c_1, \ldots, c_7 \in \{1, \ldots, 52\}$, evaluation is seven chained array dereferences with zero branches:

$$p \leftarrow \texttt{HR}\Big[\texttt{HR}\big[\ldots \texttt{HR}[\texttt{HR}[53 + c_1] + c_2] \ldots\big] + c_7\Big]. \tag{16}$$

   This achieves **60M–200M evals/s** whenever the 128 MB table fits in L3/RAM, at the cost of a 128 MB binary payload.

Evaluation Process	Core Mechanism	Table Size	Throughput	Architectural Trade-Off
21-Subset EVAL5	$\binom{7}{5} = 21$ subset loops	0 KB	0.024M / s	Zero tables; returns exact 5 winning cards for showdown display.
Bitmask RANK7FAST	Suit/rank masks + `TOP5T`	48 KB	**1.77**M / s	**75**× speedup over EVAL5; fits entirely inside L1 CPU cache.
Prime-Product Hash	$\prod_{i=1}^{5} p_{r_i}$ + perfect hash	$\approx$ 150 KB	4M–8M / s	Fast for 5-card hands; still checks $\binom{7}{5} = 21$ subsets for 7-card hands.
Quinary Combinadic	Base-5 DP hash (49,205)	2.5–5 MB	10M–115M / s	Evaluates 7 cards directly via Eq. (15) with moderate memory.
State-Graph DAG	7 chained array lookups	128 MB	60M–200M / s	Branchless 7-step pointer chase; requires 128 MB binary asset.

Table 3: Comparison of 7-card hand evaluation processes across memory footprint and throughput.

5 Process III: Equity Estimation, Range Filtering, and Runout Enumeration

5.1 Starting-Hand Isomorphism (1,326 to 169 Classes), Card Removal, and Outs

Before community cards are dealt, there are $\binom{52}{2} = \frac{52\times 51}{2} = 1,326$ distinct two-card starting hands. Because suits have identical strategic value preflop, any two hands that differ only by a permutation of suits are **strategically isomorphic**. Grouping the 1,326 combinations by rank pair and suitedness collapses them into exactly 169 **canonical starting-hand classes**:

$$\underbrace{13 \text{ Pairs}}_{13\times 6=78} + \underbrace{\binom{13}{2} = 78 \text{ Suited}}_{78\times 4=312} + \underbrace{\binom{13}{2} = 78 \text{ Offsuit}}_{78\times 12=936} = 169 \text{ classes } (1,326 \text{ combos}). \tag{17}$$

Thus, at random deal, a player is dealt a pocket pair with probability $78/1,326 = 1/17 \approx 5.88\%$, a specific pocket pair such as A A with probability $6/1,326 = 1/221 \approx 0.452\%$, a specific suited hand such as A Ks with probability $4/1,326 \approx 0.302\%$, and a specific offsuit hand such as A Ko with probability $12/1,326 \approx 0.905\%$.

On the flop (3 board cards) or turn (4 board cards), a hand that is currently behind may have O unseen **outs** remaining in the stub that will improve it to a winning hand. With $U = 52 - 2 - |B|$ unseen cards ($U = 47$ on the flop, $U = 46$ on the turn), the exact hypergeometric probability of hitting at least one out is:

$$\mathbb{P}(\text{hit turn}) = \frac{O}{47}, \qquad \mathbb{P}(\text{hit river}) = \frac{O}{46}, \qquad \mathbb{P}(\text{hit by river from flop}) = 1 - \frac{(47-O)(46-O)}{47\times 46}. \tag{18}$$

Plain-English Intuition: The Rule of 4 and 2 (and Why Engines Simulate Full Equity). Because $\frac{1}{47} \approx 2.13\%$ and $\frac{1}{46} \approx 2.17\%$, human players multiply their outs O by 4 on the flop (to estimate turn-plus-river hit probability) and by 2 on the turn (to estimate single-card river hit probability). For instance, a 9-out flush draw hits by the river with exact probability $1 - \frac{38 \times 37}{47 \times 46} = 34.97\% \approx 9 \times 4\% = 36\%$. However, counting raw outs can mislead an engine in both directions: some outs simultaneously give the opponent a better hand (**dirty outs**, such as completing a flush when the board pairs to give the opponent a full house), and sometimes the player wins via runner-runner cards or split pots without hitting a primary out. Therefore, the engine computes true **pot-share equity** rather than relying on static out counts.

5.2 The Monte Carlo Rollout Process (EQUITY) and Standard Error

Let a focal player hold $H = \{h_1, h_2\}$ on a partial board B ($|B| \in \{0, 3, 4, 5\}$) against $m \geq 1$ active opponents. Let $K = 52 - |H \cup B|$ denote the size of the unseen stub $D = \{0, \ldots, 51\} \setminus (H \cup B)$, and let $b_{\text{need}} = 5 - |B|$ denote the number of community cards still to be dealt. Each simulated runout requires drawing $k = b_{\text{need}} + 2m$ cards uniformly without replacement from D.

Rather than copying or re-initializing the K-card array D on every trial, the Monte Carlo rollout process (Algorithm 7) allocates D **once** before the loop and performs a **partial Fisher–Yates shuffle of only the first k slots** in-place across all N trials.

Algorithm 7: EQUITY — Monte Carlo pot-share estimator with in-place partial Fisher–Yates sampling

Input: focal hole cards $H = (h_1, h_2)$; current board B ($|B| \leq 5$); opponent count $m \geq 1$; trial budget N

Output: estimated pot-share equity $\hat{q} \in [0, 1]$ and standard error $\text{SE} = s/\sqrt{N}$

Build stub array $D[0 \ldots K-1] \leftarrow \{0, \ldots, 51\} \setminus (H \cup B)$, where $K = 50 - |B|$
$b_{\text{need}} \leftarrow 5 - |B|$; $k \leftarrow b_{\text{need}} + 2m$ ▷ *cards needed per trial*
Pre-fill $simH[0, 1] \leftarrow h_1, h_2$ and $simH[2 \ldots 1 + |B|] \leftarrow B$; similarly pre-fill $oppH[2 \ldots 1 + |B|] \leftarrow B$
$W_1 \leftarrow 0$; $W_2 \leftarrow 0$ ▷ *running sums $\sum w_t$ and $\sum w_t^2$*
for $t \leftarrow 1$ **to** N **do**
  for $i \leftarrow 0$ **to** $k - 1$ **do** ▷ *partial Fisher–Yates over first k slots*
    $j \leftarrow i + \lfloor U \cdot (K - i) \rfloor$, where $U \sim \text{Uniform}[0, 1)$
    Swap $D[i] \leftrightarrow D[j]$
  Copy $D[0 \ldots b_{\text{need}} - 1]$ into the remaining board slots of $simH$ and $oppH$
  $v_{\text{hero}} \leftarrow \text{RANK7FAST}(simH)$; $bestOpp \leftarrow -1$; $ties \leftarrow 0$
  for $p \leftarrow 0$ **to** $m - 1$ **do**
    $oppH[0] \leftarrow D[b_{\text{need}} + 2p]$; $oppH[1] \leftarrow D[b_{\text{need}} + 2p + 1]$
    $v_p \leftarrow \text{RANK7FAST}(oppH)$
    if $v_p > bestOpp$ **then** $bestOpp \leftarrow v_p$; $ties \leftarrow 1$ **else if** $v_p = bestOpp$ **then** $ties \leftarrow ties + 1$
  $w_t \leftarrow 1$ **if** $v_{\text{hero}} > bestOpp$, **else** $\frac{1}{ties+1}$ **if** $v_{\text{hero}} = bestOpp$, **else** 0
  $W_1 \leftarrow W_1 + w_t$; $W_2 \leftarrow W_2 + w_t^2$
$\hat{q} \leftarrow W_1/N$; $s^2 \leftarrow \max(0,\ W_2/N - \hat{q}^2)$; $\text{SE} \leftarrow \sqrt{s^2/N}$
return $(\hat{q}, \text{SE})$

Proposition 5.1 (Correctness of Non-Resetting Partial Fisher–Yates Across Trials)**.** *In Algorithm 7, mutating D in-place across trials $t = 1, \ldots, N$ without resetting D to sorted order between trials still produces an independent, strictly uniform k-tuple of distinct cards from D on every trial.*

Proof. At the start of any trial t, because every step of prior trials only ever swapped elements within D, the array $D[0 \dots K-1]$ is a bijection (permutation) of the fixed set of K unseen cards. Executing k steps of Fisher–Yates on any initial bijection of K distinct elements maps each of the $K(K-1)\cdots(K-k+1)$ ordered k-tuples into $D[0 \dots k-1]$ with probability $\prod_{i=0}^{k-1} \frac{1}{K-i}$, independent of the initial permutation of D. □

Proposition 5.2 (Exact Sample Variance over Fractional Split-Pot Outcomes). *Let $w_t \in [0,1]$ be the fractional pot share won on trial t. The sample variance $s^2 = \frac{W_2}{N} - \hat{q}^2$ satisfies $s^2 \le \hat{q}(1-\hat{q})$, with strict inequality $s^2 < \hat{q}(1-\hat{q})$ whenever split pots ($0 < w_t < 1$) occur.*

Proof. For every $w_t \in [0,1]$, we have $w_t^2 \le w_t$, with strict inequality $w_t^2 < w_t$ whenever $0 < w_t < 1$. Hence $\frac{W_2}{N} = \frac{1}{N}\sum_{t=1}^{N} w_t^2 \le \frac{1}{N}\sum_{t=1}^{N} w_t = \hat{q}$, so $s^2 = \frac{W_2}{N} - \hat{q}^2 \le \hat{q} - \hat{q}^2 = \hat{q}(1-\hat{q})$. On chopped boards where every trial ties ($w_t = 1/2$), the Bernoulli formula $\sqrt{\hat{q}(1-\hat{q})/N}$ erroneously reports $\mathrm{SE} = 0.5/\sqrt{N}$, whereas Algorithm 7 yields the exact $s^2 = 0.25 - 0.25 = 0 \implies \mathrm{SE} = 0$. □

5.3 The Range-Conditioned Rejection-Sampling Process (EQUITYVSRANGE)

Uniform Monte Carlo (EQUITY) assumes that an opponent is equally likely to hold 7♠ 2♢ as A♠ A♡. Once an opponent voluntarily calls or raises preflop, their range is heavily skewed toward stronger starting hands. To condition postflop equity on the opponent's action history, the range-conditioned rollout process (EQUITYVSRANGE, Algorithm 8) performs **rejection sampling** on the opponent's 2-card hand using a precomputed 52×52 table `CHEN_PAIR`$[c_1, c_2]$ of starting-hand Chen scores: if the sampled pair (c_1, c_2) has `CHEN_PAIR`$[c_1, c_2] < c_{\text{floor}}$, it is rejected and re-drawn before completing the board runout.

Algorithm 8: EQUITYVSRANGE — rejection-sampled Monte Carlo equity against an action-conditioned range floor

Input: hero hole cards H; board B; opponent Chen floor c_{floor}; trial budget N
Output: conditional equity $\hat{q}(H \mid \text{Chen}(H_{\text{opp}}) \ge c_{\text{floor}})$, or **nil** if sample size < 30

Build stub $D[0 \dots K-1] \leftarrow \{0, \dots, 51\} \setminus (H \cup B)$; $b_{\text{need}} \leftarrow 5 - |B|$
$W_1 \leftarrow 0$; $n_{\text{acc}} \leftarrow 0$; $n_{\text{att}} \leftarrow 0$; $N_{\max} \leftarrow 14N$
while $n_{\text{acc}} < N \wedge n_{\text{att}} < N_{\max}$ **do**
  $n_{\text{att}} \leftarrow n_{\text{att}} + 1$
  Draw 2 opponent cards $D[0], D[1]$ via 2 Fisher–Yates swaps on $D[0 \dots K-1]$
  if `CHEN_PAIR`$[D[0], D[1]] < c_{\text{floor}}$ **then continue** ▷ *reject hands below range floor*
  Draw b_{need} board cards $D[2 \dots 1 + b_{\text{need}}]$ via Fisher–Yates swaps on $D[2 \dots K-1]$
  $v_{\text{hero}} \leftarrow \text{RANK7FAST}(H \cup B \cup D[2 \dots 1 + b_{\text{need}}])$
  $v_{\text{opp}} \leftarrow \text{RANK7FAST}(\{D[0], D[1]\} \cup B \cup D[2 \dots 1 + b_{\text{need}}])$
  $W_1 \leftarrow W_1 + (1 \text{ if } v_{\text{hero}} > v_{\text{opp}},\ 0.5 \text{ if } v_{\text{hero}} = v_{\text{opp}},\ \text{else } 0)$
  $n_{\text{acc}} \leftarrow n_{\text{acc}} + 1$
return W_1 / n_{acc} **if** $n_{\text{acc}} \ge 30$ **else nil**

Example 5.1 (Numerical Impact of Conditioning on the Opponent's Range)**.** Suppose a player holds Pocket Tens (T♠ T♡) on a flop of K♠ Q♢ 5♣.

- Against a **uniform random hand** (EQUITY, $c_{\text{floor}} = -\infty$): T♠ T♡ has 70.0% **equity**.
- Against a **single-raised range** (EQUITYVSRANGE, $c_{\text{floor}} = 6.1$): hands containing a King or Queen make up a large share of the range, dropping T♠ T♡ equity to 46.9%.
- Against a **3-bet range** (EQUITYVSRANGE, $c_{\text{floor}} = 9.1$): the opponent holds almost exclusively A A, K K, Q Q, J J, T T, A K, A Qs, dropping T♠ T♡ equity to 18.1%.

5.4 The Exact Turn/River Enumeration and Next-Card Vulnerability Process

When the board reaches the river ($|B| = 5$) or turn ($|B| = 4$) in a heads-up pot ($m = 1$), the remaining sample space is small enough to replace random simulation with **exact deterministic enumeration** (SE $= 0$, Algorithm 9):

- **River ($|B| = 5$, 45 unseen cards):** Enumerating all $\binom{45}{2} = \frac{45\times44}{2} = \mathbf{990}$ possible opponent two-card holdings computes exact river equity in < 0.6 ms.
- **Turn ($|B| = 4$, 46 unseen cards):** For each of the 46 possible river cards $c \in D$, enumerating all $\binom{45}{2} = 990$ opponent holdings requires $46 \times 990 = \mathbf{45,540}$ evaluations (≈ 24 ms). Averaging the 46 exact conditional equities $q(c)$ yields exact turn equity $q_{\text{base}} = \frac{1}{46}\sum_{c\in D} q(c)$ with zero sampling error (SE $= 0$) and classifies every unseen river card c as a **helpful card** ($q(c) - q_{\text{base}} > 0.03$, i.e., an *effective out*), a **scare card** ($q(c) - q_{\text{base}} < -0.03$), or **neutral**.
- **Flop ($|B| = 3$, 47 unseen cards) or Multi-Way Turn ($m \geq 2$):** Runs Monte Carlo rollouts ($N_{\text{base}} = 500$ trials for baseline equity and $N_{\text{per}} = 200$ trials per candidate card $c \in D$) and widens the classification threshold dynamically to the 2σ **noise floor** of the difference between two independent Monte Carlo estimators:

$$\tau = \max\left(0.03,\ 2\sqrt{\text{SE}_{\text{base}}^2 + \text{SE}_c^2}\right) \approx 0.036\text{–}0.070, \tag{19}$$

ensuring that random sampling jitter never mislabels a neutral card as an out or scare card.

Algorithm 9: NEXTCARDSCAN — exact turn/river enumeration and dynamic 2σ next-card vulnerability scanner

Input: hole cards $H = (h_1, h_2)$; board B ($|B| \in \{3,4\}$); active opponents $m \geq 1$
Output: baseline equity q_{base}, helpful outs count n_+, scare card count n_-, and per-rank deltas

$D \leftarrow \{0,\ldots,51\} \setminus (H \cup B)$
if $|B| = 4 \wedge m = 1$ **then** ▷ *exact deterministic enumeration over* $46 \times \binom{45}{2} = 45,540$ *deals*
  foreach candidate river card $c \in D$ **do**
    $B' \leftarrow B \cup \{c\}$; $R \leftarrow D \setminus \{c\}$; $v_{\text{hero}} \leftarrow \text{RANK7FAST}(H \cup B')$; $W \leftarrow 0$
    for $i \leftarrow 0$ **to** 43 **do for** $j \leftarrow i+1$ **to** 44 **do**
      $v_{\text{opp}} \leftarrow \text{RANK7FAST}(\{R_i, R_j\} \cup B')$
      $W \leftarrow W + (1 \text{ if } v_{\text{hero}} > v_{\text{opp}},\ 0.5 \text{ if } v_{\text{hero}} = v_{\text{opp}},\ \text{else } 0)$
    $q(c) \leftarrow W/990$; $\text{SE}_c \leftarrow 0$
  $q_{\text{base}} \leftarrow \frac{1}{46}\sum_{c\in D} q(c)$; $\text{SE}_{\text{base}} \leftarrow 0$
else ▷ *Monte Carlo rollout with dynamic* 2σ *noise filter*
  $(q_{\text{base}}, \text{SE}_{\text{base}}) \leftarrow \text{EQUITY}(H, B, m, N_{\text{base}})$
  foreach $c \in D$ **do** $(q(c), \text{SE}_c) \leftarrow \text{EQUITY}(H, B \cup \{c\}, m, N_{\text{per}})$
$n_+ \leftarrow 0$; $n_- \leftarrow 0$
foreach $c \in D$ **do**
  $\tau_c \leftarrow \max\left(0.03,\ 2\sqrt{\text{SE}_{\text{base}}^2 + \text{SE}_c^2}\right)$; $\Delta_c \leftarrow q(c) - q_{\text{base}}$
  if $\Delta_c > \tau_c$ **then** $n_+ \leftarrow n_+ + 1$ **else if** $\Delta_c < -\tau_c$ **then** $n_- \leftarrow n_- + 1$
return $(q_{\text{base}}, n_+, n_-)$

6 Process IV: The Betting State Machine and Turn-Order Process

6.1 Player Chip Accounting and the Dead-Button Blind Process (STARTHAND)

To prevent chip-accounting discrepancies across multi-street betting and side-pot settlement, the state machine maintains three distinct integer counters on every player record p:

- $p.\texttt{stack} \geq 0$: Chips currently sitting in front of player p available to be wagered.
- $p.\texttt{bet} \geq 0$: Chips wagered by player p **on the current street only** (reset to 0 at the start of the flop, turn, and river).
- $p.\texttt{committed} \geq 0$: Cumulative chips wagered by player p **across the entire hand** (never reset until the hand ends; only reduced if an uncalled overbet is refunded).

Whenever player p puts δ chips into the pot, all three counters update atomically via the conservation rule:

$$p.\texttt{stack} \mathrel{-}= \delta, \quad p.\texttt{bet} \mathrel{+}= \delta, \quad p.\texttt{committed} \mathrel{+}= \delta, \quad \texttt{pot} \mathrel{+}= \delta. \tag{20}$$

When a player busts ($p.\texttt{stack} = 0$ at the end of a hand), naively advancing the dealer button to the next funded seat can cause a surviving player to skip the big blind entirely. Under official Poker Tournament Directors Association (TDA) **dead-button / forward-moving blind rules**, the big blind (`bbSeat`) is anchored first by advancing strictly clockwise from the previous hand's big blind seat; only then are the small blind and button assigned (Algorithm 10). Furthermore, if the big blind player has a short stack ($p_{\texttt{bb}}.\texttt{stack} < \texttt{bb}$), `currentBet` is still initialized to the full `bb` (not the short stack's all-in amount) so the next player cannot limp in for less than a full big blind.

Algorithm 10: STARTHAND — dead-button blind advancement, short-blind bring-in, and preflop initialisation

Input: previous blind seat `bbSeat`; funded seat list $E = (e_0, \ldots, e_{k-1})$ with $k \geq 2$

Output: updated $(\texttt{btnSeat}, \texttt{sbSeat}, \texttt{bbSeat})$, posted blinds, and first actor `actingIdx`

if $\texttt{bbSeat} = -1$ **then** $\texttt{btnSeat} \leftarrow e_0$; $\texttt{sbSeat} \leftarrow (e_0 \text{ if } k = 2 \text{ else } e_1)$; $\texttt{bbSeat} \leftarrow (e_1 \text{ if } k = 2 \text{ else } e_2)$

else

  $\texttt{bbSeat} \leftarrow \text{NEXTFUNDEDSEATCLOCKWISE}(\texttt{bbSeat})$ ▷ *anchor big blind forward*

  if $k = 2$ **then** $\texttt{sbSeat} \leftarrow \text{PREVFUNDEDSEAT}(\texttt{bbSeat})$; $\texttt{btnSeat} \leftarrow \texttt{sbSeat}$ ▷ *heads-up: BTN is SB*

  else $\texttt{sbSeat} \leftarrow \text{PREVFUNDEDSEAT}(\texttt{bbSeat})$; $\texttt{btnSeat} \leftarrow \text{PREVFUNDEDSEAT}(\texttt{sbSeat})$

$\text{POSTBLIND}(p_{\texttt{sb}}, \texttt{sb})$; $\text{POSTBLIND}(p_{\texttt{bb}}, \texttt{bb})$

$\texttt{currentBet} \leftarrow \max(\texttt{bb}, p_{\texttt{bb}}.\texttt{bet})$; $\texttt{lastRaise} \leftarrow \texttt{bb}$ ▷ *full BB bring-in even if BB is short*

$\texttt{actingIdx} \leftarrow \text{FINDNEXTACTOR}(\texttt{bbSeat})$ ▷ *UTG acts first preflop (SB/BTN in heads-up)*

6.2 The Turn-Order and Round-Completion Process (FINDNEXTACTOR)

After any player acts (or at the start of a street), how does the state machine decide who acts next or whether the betting round is over? The turn-order process (FINDNEXTACTOR, Algorithm 11) scans seats clockwise starting after seat *from*. A non-folded, non-all-in player p requires an action if and only if they satisfy the **Pending Action Disjunction**:

$$\text{NeedsAction}(p) \iff \neg p.\texttt{folded} \wedge \neg p.\texttt{allIn} \wedge \big(\neg p.\texttt{hasActed} \vee p.\texttt{bet} < \texttt{currentBet}\big). \tag{21}$$

Algorithm 11: FINDNEXTACTOR — determine next seat to act or signal end of betting street

Input: seat index *from* (-1 at street start); player array $P[0 \dots N_p - 1]$; street bet `currentBet`
Output: next seat index $idx \in \{0, \dots, N_p - 1\}$, or -1 if the betting street is complete

$n_{\text{live}} \leftarrow |\{p \in P : \neg p.\texttt{folded}\}|$; $\quad n_{\text{canAct}} \leftarrow |\{p \in P : \neg p.\texttt{folded} \wedge \neg p.\texttt{allIn}\}|$
if $n_{\text{live}} \leq 1$ **then return** -1 ▷ *everyone else folded; hand ends immediately*
if $n_{\text{canAct}} = 0$ **then return** -1 ▷ *all remaining players are all-in; run out board*
for $k \leftarrow 1$ **to** N_p **do**
  $idx \leftarrow (from + k) \bmod N_p$; $\quad p \leftarrow P[idx]$
  if $p.\texttt{folded} \vee p.\texttt{allIn}$ **then continue**
  if $\neg p.\texttt{hasActed} \vee p.\texttt{bet} < \texttt{currentBet}$ **then return** idx
return -1 ▷ *every active player has acted and matched* `currentBet`

6.3 The Action Application and Cumulative Short All-In Reopening Process

A subtle rule in No-Limit Hold'em concerns **short all-in raises**. Suppose the big blind is 20, Player A bets 100 (`lastRaise` $= 100$), and Player B moves all-in for 140 total (an increase of only $40 < 100$). When the action returns to Player A, Player A owes 40 more chips ($p_A.\texttt{bet} = 100 < 140$), so FINDNEXTACTOR gives Player A a turn. However, because Player B's all-in increase (40) was less than the full raise increment (100), **Player A is only permitted to call the** 40 **or fold; Player A may not re-raise!** What if Player C also goes all-in for 210 total after Player B? Individually, neither $140 - 100 = 40$ nor $210 - 140 = 70$ is a full 100 raise, but **cumulatively** the bet has risen from 100 to 210 ($210 - 100 = 110 \geq 100$), which **does** reopen raising for Player A under Poker TDA Rules.

To enforce both individual and cumulative reopening cleanly, the state machine stores two variables on eachplayer p when they act: $p.\texttt{actedAtBet}$ (the street's `currentBet` after their action) and $p.\texttt{minRaiseSeen}$ (the required full raise increment `lastRaise` in effect at that moment). Whenever a full raise occurs ($\Delta = \tau - \texttt{currentBet} \geq \texttt{lastRaise}$), `lastRaise` updates to Δ and every other player's `hasActed` flag is reset to **false**; on a short all-in ($\Delta < \texttt{lastRaise}$), `lastRaise` and `hasActed` are left untouched, and CANREOPEN (Algorithm 12) checks whether $\texttt{currentBet} - p.\texttt{actedAtBet} \geq p.\texttt{minRaiseSeen}$.

Algorithm 12: APPLYACTIONANDREOPEN — state transition for player actions and cumulative short all-in reopening check

Input: player p; action (a, τ); state $(\texttt{currentBet}, \texttt{lastRaise}, \texttt{raises})$
Output: updated betting state and next actor

Function CANREOPEN(p):
  if $p.\texttt{stack} \leq \max(0, \texttt{currentBet} - p.\texttt{bet})$ **then return false** ▷ *no chips beyond a call*
  if $\neg p.\texttt{hasActed}$ **then return true** ▷ *has not acted since last full raise*
  return $(\texttt{currentBet} - p.\texttt{actedAtBet}) \geq p.\texttt{minRaiseSeen}$ ▷ *cumulative short all-ins ≥ full raise*
Procedure APPLYACTION(p, a, τ):
  if $a =$ FOLD **then** $p.\texttt{folded} \leftarrow$ **true**; $p.\texttt{hasActed} \leftarrow$ **true**
  else if $a =$ CHECK **then** $p.\texttt{hasActed} \leftarrow$ **true**; $p.\texttt{actedAtBet} \leftarrow \texttt{currentBet}$; $p.\texttt{minRaiseSeen} \leftarrow \texttt{lastRaise}$
  else if $a =$ CALL **then**
    $\delta \leftarrow \min(p.\texttt{stack}, \max(0, \texttt{currentBet} - p.\texttt{bet}))$
    Transfer δ chips via Eq. (20); **if** $p.\texttt{stack} = 0$ **then** $p.\texttt{allIn} \leftarrow$ **true**
    $p.\texttt{hasActed} \leftarrow$ **true**; $p.\texttt{actedAtBet} \leftarrow \texttt{currentBet}$; $p.\texttt{minRaiseSeen} \leftarrow \texttt{lastRaise}$
  else if $a =$ RAISE **then**
    $\tau_{\max} \leftarrow p.\texttt{bet} + p.\texttt{stack}$; $\tau_{\min} \leftarrow \min(\tau_{\max}, \max(\texttt{currentBet} + \texttt{lastRaise}, \texttt{bb}))$
    $\tau^\star \leftarrow \min(\tau_{\max}, \max(\tau_{\min}, \tau))$; $\delta \leftarrow \tau^\star - p.\texttt{bet}$; $\Delta \leftarrow \tau^\star - \texttt{currentBet}$
    Transfer δ chips via Eq. (20); **if** $p.\texttt{stack} = 0$ **then** $p.\texttt{allIn} \leftarrow$ **true**
    if $\Delta \geq \texttt{lastRaise}$ **then** ▷ *full raise: update* lastRaise *and reopen table*
      $\texttt{lastRaise} \leftarrow \Delta$; $\texttt{raises} \leftarrow \texttt{raises} + 1$
      foreach $q \neq p$ **with** $\neg q.\texttt{folded} \wedge \neg q.\texttt{allIn}$ **do** $q.\texttt{hasActed} \leftarrow$ **false**
    $\texttt{currentBet} \leftarrow \tau^\star$; $p.\texttt{hasActed} \leftarrow$ **true**; $p.\texttt{actedAtBet} \leftarrow \tau^\star$; $p.\texttt{minRaiseSeen} \leftarrow \texttt{lastRaise}$

7 Process V: Uncalled Bet Refund, Side-Pot Slicing, and Showdown Settlement

7.1 The Uncalled Bet Refund Process (REFUNDUNCALLED)

Before any pots are sliced or cards are compared at showdown, if the player $p_{\max}$ with the strictly highest total commitment $c_1 = \max_p p.\texttt{committed}$ has committed more than the second-highest commitment $c_2 = \max_{p \neq p_{\max}} p.\texttt{committed}$, no opponent matched the excess $\Delta_{\text{uncalled}} = c_1 - c_2$. Under universal poker rules, those Δ_{uncalled} chips are **not part of any pot**; they are immediately returned to $p_{\max}$ before showdown (Algorithm 13):

$$p_{\max}.\texttt{committed} \leftarrow c_2, \qquad p_{\max}.\texttt{stack} \leftarrow p_{\max}.\texttt{stack} + (c_1 - c_2), \qquad \texttt{pot} \leftarrow \texttt{pot} - (c_1 - c_2). \tag{22}$$

Algorithm 13: REFUNDUNCALLED — return unmatched excess commitment before pot decomposition

Input: player array P
Output: excess chips Δ_{uncalled} returned to the sole highest committer

Sort committers $C \leftarrow \{p \in P : p.\texttt{committed} > 0\}$ in descending order of $p.\texttt{committed}$
if $|C| = 0$ **then return**
$c_1 \leftarrow C_0.\texttt{committed}; \quad c_2 \leftarrow (C_1.\texttt{committed}$ if $|C| \geq 2$ else $0)$
if $c_1 > c_2$ **then**
  $\Delta \leftarrow c_1 - c_2$
  $C_0.\texttt{committed} \leftarrow c_2; \quad C_0.\texttt{stack} \leftarrow C_0.\texttt{stack} + \Delta; \quad \texttt{pot} \leftarrow \texttt{pot} - \Delta$

7.2 The Horizontal Side-Pot Slicing Process (COMPUTEPOTS)

When multiple players are all-in for different amounts, how does an engine split the pot into a Main Pot and one or more Side Pots? The horizontal commitment-slicing process (COMPUTEPOTS, Algorithm 14) collects all distinct positive commitment values $0 = L_0 < L_1 < L_2 < \cdots < L_k$ across all players who put chips into the pot (including players who later folded). For each horizontal band $[L_{j-1}, L_j]$, every player p contributes the slice of their commitment that falls inside that band:

$$\gamma(p, L_{j-1}, L_j) = \max\Big(0,\ \min(p.\texttt{committed}, L_j) - \min(p.\texttt{committed}, L_{j-1})\Big). \tag{23}$$

The total chips in band j are $B_j = \sum_{p \in P} \gamma(p, L_{j-1}, L_j)$, and the set of players **eligible to win** band j is:

$$E_j = \big\{p \in P : \neg p.\texttt{folded} \ \wedge\ p.\texttt{committed} \geq L_j\big\}. \tag{24}$$

Algorithm 14: COMPUTEPOTS — slice multi-way commitments into horizontal bands, merge identical eligibility sets, and guard orphaned bands

Input: player array P; total pot $\texttt{pot}$
Output: ordered list of pots $\mathcal{P} = \big((amount_1, E_1), \ldots, (amount_r, E_r)\big)$

Extract strictly increasing positive commitment levels $L_1 < L_2 < \cdots < L_k$ from $\{p.\texttt{committed} > 0\}$
$\mathcal{P} \leftarrow [\,]; \quad prev \leftarrow 0; \quad lastEligible \leftarrow \{p.\texttt{id} : \neg p.\texttt{folded}\}$
for $j \leftarrow 1$ **to** k **do**
  $B_j \leftarrow \sum_{p \in P} \gamma(p, prev, L_j)$ via Eq. (23)
  $E_j \leftarrow \{p.\texttt{id} : \neg p.\texttt{folded} \wedge p.\texttt{committed} \geq L_j\}$
  if $E_j \neq \emptyset$ **then** $lastEligible \leftarrow E_j$ **else** $E_j \leftarrow lastEligible$ ▷ *orphaned-band backstop*
  if $B_j > 0$ **then**
    if $\mathcal{P} \neq [\,] \wedge \mathcal{P}_{\text{last}}.E = E_j$ **then** $\mathcal{P}_{\text{last}}.amount \mathrel{+}= B_j$ ▷ *merge adjacent bands with same E_j*
    else append (B_j, E_j) to $\mathcal{P}$
  $prev \leftarrow L_j$
return $\mathcal{P}$

Proposition 7.1 (Chip Conservation and Orphaned-Band Backstop in COMPUTEPOTS)**.** *Across all bands $j = 1, \ldots, k$, the sum of band amounts equals total committed chips: $\sum_{j=1}^{k} B_j = \sum_{p \in P} p.\texttt{committed}$. Furthermore, if the highest committer(s) at level L_k have folded so that $E_k = \emptyset$, Line 6 assigns B_k to the highest non-empty eligibility set lastEligible, guaranteeing that every chip in* pot *has at least one eligible winner.*

Proof. For each individual player p with $p.\texttt{committed} = L_m$, the telescoping sum of Eq. (23) across $j = 1, \ldots, k$ is $\sum_{j=1}^{m}(L_j - L_{j-1}) + \sum_{j=m+1}^{k} 0 = L_m - 0 = p.\texttt{committed}$. Summing over all $p \in P$ proves $\sum_{j=1}^{k} B_j = \sum_{p \in P} p.\texttt{committed}$. Because E_j is monotonically non-increasing ($E_1 \supseteq E_2 \supseteq \cdots \supseteq E_k$) and at least one player remains un-folded at showdown, *lastEligible* is always non-empty; thus substituting *lastEligible* whenever $E_j = \emptyset$ prevents dead folded over-commitments from vanishing. □

Example 7.1 (Worked 4-Player Multi-Way Side-Pot Decomposition)**.** Suppose four players commit the following amounts during a hand:

- **Player A:** All-in for 100 (un-folded).
- **Player B:** Committed 200, then **folded** later in the hand.
- **Player C:** All-in for 350 (un-folded).
- **Player D:** Committed 350 (un-folded).

The distinct commitment levels are $L_1 = 100$, $L_2 = 200$, and $L_3 = 350$, with total pot $100 + 200 + 350 + 350 = 1,000$:

1. **Band 1 (**$0 \to 100$**):** All 4 players contribute 100 each $\implies$ $B_1 = 400$. Eligible winners: $E_1 = \{A, C, D\}$ (since B folded). This forms the **Main Pot of** 400 contested by $\{A, C, D\}$.
2. **Band 2 (**$100 \to 200$**):** Players B, C, D each contribute $200 - 100 = 100$ $\implies$ $B_2 = 300$. Eligible winners: $E_2 = \{C, D\}$.
3. **Band 3 (**$200 \to 350$**):** Players C, D each contribute $350 - 200 = 150$ $\implies$ $B_3 = 300$. Eligible winners: $E_3 = \{C, D\}$. Because $E_3 = E_2 = \{C, D\}$, Line 8 merges Band 2 and Band 3 into a single **Side Pot 1 of** $300 + 300 = 600$ contested by $\{C, D\}$.

7.3 The Showdown Evaluation and Odd-Chip Settlement Process (AWARDPOTS)

Once the pots $\mathcal{P} = \big((amount_1, E_1), \ldots, (amount_r, E_r)\big)$ are constructed, the showdown settlement process (Algorithm 15) evaluates each un-folded player's 7-card hand once via BEST7$(p.H \cup B)$ and awards each pot independently. If $m \geq 2$ eligible players tie for the highest score in a pot of size A, each tied winner receives $\lfloor A/m \rfloor$ chips, and the $r = A \bmod m$ indivisible **odd chips** (1 chip each) are awarded to the first r tied winners in **clockwise seat order starting immediately left of the dealer button** ($\texttt{btnSeat} + 1$), matching Poker TDA Rule 20.

Algorithm 15: AWARDPOTS — evaluate showdown hands, settle main and side pots, and distribute odd chips clockwise

Input: pots $\mathcal{P}$; un-folded players L; board B; dealer button seat $\texttt{btnSeat}$
Output: all pot chips transferred to winning player stacks ($\texttt{pot} \leftarrow 0$)

1 **foreach** $p \in L$ **do** $(p.\texttt{score}, p.\texttt{best5}) \leftarrow$ BEST7$(p.H \cup B)$
2 **foreach** pot $(amount, E) \in \mathcal{P}$ **do**
3 | $v^\star \leftarrow \max_{p \in E} p.\texttt{score}$; $W \leftarrow \{p \in E : p.\texttt{score} = v^\star\}$
4 | Sort tied winners $W = (w_0, \ldots, w_{m-1})$ by clockwise distance $(w.\texttt{id} - \texttt{btnSeat} - 1) \bmod N_p$
5 | $q \leftarrow \lfloor amount/m \rfloor$; $r \leftarrow amount \bmod m$
6 | **for** $k \leftarrow 0$ **to** $m - 1$ **do** $w_k.\texttt{stack} \leftarrow w_k.\texttt{stack} + q + (1 \text{ if } k < r \text{ else } 0)$
7 $\texttt{pot} \leftarrow 0$

8 Process VI: Strategic Decision Policy, Nash Push/Fold, and Solver Processes

8.1 Player Archetypes and Difficulty Interpolation

To produce distinct, believable table personalities while scaling cleanly across difficulty settings, each computer opponent is parameterized by a 3-dimensional **persona vector** (θ, α, β):

- $\theta \in [0.02, 0.10]$ (**Tightness**): Minimum equity edge required above raw pot odds before calling, plus preflop hand-selection selectivity.
- $\alpha \in [0.38, 0.72]$ (**Aggression**): Propensity to raise rather than call or check with strong value hands.
- $\beta \in [0.08, 0.30]$ (**Bluff Frequency**): Propensity to bet or raise with weak holdings or semi-bluff draws.

Table 4 lists the five classic poker archetypes. As difficulty increases from CASUAL ($\lambda = 0$) to SOLID ($\lambda = 0.35$) to SHARP ($\lambda = 0.70$), each seat's parameters are linearly pulled toward a disciplined baseline $(\theta_0, \alpha_0, \beta_0) = (0.04, 0.58, 0.14)$:

$$(\theta_{\text{eff}}, \alpha_{\text{eff}}, \beta_{\text{eff}}) = (1 - \lambda)(\theta, \alpha, \beta) + \lambda(\theta_0, \alpha_0, \beta_0). \tag{25}$$

Archetype	Code	θ	α	β	Behavioural Signature at the Table
Tight-Aggressive	TAG	0.06	0.62	0.14	Selective preflop; enters pots primarily via raises and value bets strongly.
Loose-Aggressive	LAG	0.02	0.72	0.30	Plays wide ranges, applies maximum bet pressure, and bluffs frequently.
Calling Station	CALL	0.02	0.38	0.12	Calls too wide preflop and postflop; rarely raises without a monster hand.
Nit (Ultra-Tight)	NIT	0.10	0.45	0.08	Folds marginal hands early; enters pots almost exclusively with premium holdings.
Balanced Solid	SOLID	0.04	0.58	0.14	Disciplined baseline toward which all archetypes converge at higher difficulty.

Table 4: Opponent persona parameters (θ, α, β) and their strategic tendencies.

8.2 The Preflop Heuristic Valuation Process (CHENSCORE)

In deep-stacked or multi-way preflop pots ($b_{\text{eff}} > 20$ big blinds or more than 2 live players), evaluating all 169 starting-hand classes quickly requires a closed-form heuristic. Bill Chen's starting-hand formula (CHENSCORE, Algorithm 16) scores any two hole cards (c_1, c_2) in $O(1)$ time by awarding base points to the higher card, doubling the score if paired (minimum 5), adding 2 points if suited, subtracting a gap penalty for disconnected ranks, and adding 1 bonus point for compact low-gap connectors that can make the nut straight.

Algorithm 16: CHENSCORE — closed-form preflop valuation of a two-card starting hand

Input: two hole cards c_1, c_2 with ranks $h = \max(\rho(c_1), \rho(c_2))$ and $\ell = \min(\rho(c_1), \rho(c_2))$
Output: Chen score $s \in [-1, 20]$

$s \leftarrow 10$ **if** $h = 12$ (A), 8 **if** $h = 11$ (K), 7 **if** $h = 10$ (Q), 6 **if** $h = 9$ (J), **else** $(h+2)/2$
if $h = \ell$ **then return** $\max(5, 2s)$ ▷ *pocket pair (*A A $= 20$, K K $= 16, \ldots, 2\,2 = 5$*)*
if $\sigma(c_1) = \sigma(c_2)$ **then** $s \leftarrow s + 2$ ▷ *suited bonus*
$g \leftarrow h - \ell - 1$ ▷ *rank gap between the two cards*
$s \leftarrow s - \big(0 \text{ if } g = 0,\ 1 \text{ if } g = 1,\ 2 \text{ if } g = 2,\ 4 \text{ if } g = 3,\ \text{else } 5\big)$
if $g \le 1 \wedge h < 10$ **then** $s \leftarrow s + 1$ ▷ *straight-connectivity bonus below Queen*
return s

8.3 The Short-Stack Nash Push/Fold Equilibrium Process (PUSHFOLD)

When effective stacks drop to $B_{\text{eff}} \le 20$ big blinds in a heads-up pot ($n_{\text{live}} = 2$), postflop play shrinks and heuristic scoring becomes exploitable. Instead, the engine uses a **precomputed** 169×19 **Heads-Up Nash Equilibrium Push/Fold Table** (`PF_TABLE`) solved offline via **Regret Matching** over 2,500 self-play iterations across the exact 169×169 preflop equity matrix $E[i, j]$ (incorporating card-removal weights $W[i, j]$).

In the push/fold subgame at stack depth $B \in \{2, 3, \ldots, 20\}$ big blinds (with small blind 0.5 bb and big blind 1.0 bb):

- If the Small Blind **folds**, the Small Blind's payoff is -0.5 bb.
- If the Small Blind **pushes all-in** with hand class i and the Big Blind **folds**, the Small Blind wins $+1.0$ bb (netting the Big Blind's posted 1 bb).
- If the Small Blind **pushes all-in** with hand class i and the Big Blind **calls** with hand class j, the total pot is $2B$ bb and the Small Blind's expected net payoff is $E[i, j] \cdot (2B) - B = B(2E[i, j] - 1)$.

Thus, given the Big Blind's calling probabilities $c_B(j) \in [0, 1]$ and the Small Blind's pushing probabilities $s_B(i) \in [0, 1]$, the exact expected values are:

$$\text{EV}_{\text{SB}}(\text{Push } i \mid c_B) = \frac{\sum_{j=0}^{168} W[i, j] \Big((1 - c_B(j)) \cdot 1.0 + c_B(j) \cdot B(2E[i, j] - 1) \Big)}{\sum_{j=0}^{168} W[i, j]}, \tag{26}$$

$$\text{EV}_{\text{BB}}(\text{Call } j \mid s_B) = \frac{\sum_{i=0}^{168} W[i, j]\, s_B(i) \Big((1 - E[i, j]) \cdot (2B) - (B - 1) \Big)}{\sum_{i=0}^{168} W[i, j]\, s_B(i)}. \tag{27}$$

At equilibrium, the Small Blind pushes whenever $\text{EV}_{\text{SB}}(\text{Push } i) > -0.5$, and the Big Blind calls whenever $\text{EV}_{\text{BB}}(\text{Call } j) > -1.0$. Table 5 shows the resulting combo-weighted push and call frequencies across stack depths, and Algorithm 17 specifies the $O(1)$ runtime lookup process.

Algorithm 17: PUSHFOLD — $O(1)$ lookup in the 169×19 heads-up Nash push/fold equilibrium table

Input: hole cards (c_1, c_2); effective stack in big blinds b_{eff}; role $isPusher \in \{\textbf{true}, \textbf{false}\}$
Output: equilibrium probability $\pi^\star \in [0, 1]$ of shoving (SB) or calling an all-in (BB)

$r_1 \leftarrow \rho(c_1)$; $r_2 \leftarrow \rho(c_2)$; $h \leftarrow 12 - \max(r_1, r_2)$; $\ell \leftarrow 12 - \min(r_1, r_2)$
$(row, col) \leftarrow (h, \ell)$ **if** $r_1 = r_2$ **else** (h, ℓ) **if** $\sigma(c_1) = \sigma(c_2)$ **else** (ℓ, h) ▷ *map to* 13×13 *grid*
$idx \leftarrow 13 \cdot row + col \in \{0, \ldots, 168\}$
$b \leftarrow \max\big(2,\ \min(20, \text{round}(b_{\text{eff}}))\big) - 2 \in \{0, \ldots, 18\}$
return `PF_TABLE`$[idx][b][isPusher ? 0 : 1]/9$

Effective Stack (B)	SB Shove %	BB Call %	Representative Equilibrium Boundary Hands
20 bb	42.9%	22.0%	SB shoves 22+, A2s+, A7o+, K9s+, Q9s+, J9s+, T8s+, 98s; BB calls 44+, A8s+, ATo+, KJs+.
15 bb	51.2%	28.3%	SB widens to A2o+, K5s+, Q8s+, 87s, 76s; BB calls 33+, A5s+, A8o+, KTs+, KQo.
10 bb	61.3%	40.4%	SB shoves 61% of all hands (including K2s+, Q5s+, J7s+, 65s); BB calls 22+, A2s+, A4o+, K7s+, Q9s+.
5 bb	75.1%	68.2%	High dead-money leverage (1.5 bb in blinds vs. 5 bb stack); BB defends over two-thirds of hands.
2 bb	94.5%	100.0%	Getting 3 : 1 pot odds (1 bb to call into a 3 bb pot, needing only 25% equity), BB calls 100% of hands.

Table 5: Combo-weighted Heads-Up Nash Equilibrium shove and call percentages across stack depths.

8.4 The Preflop and Postflop Action Selection Processes

In multi-way or deep-stacked preflop situations, the preflop policy (PREFLOPDECISION) compares CHENSCORE$(p.H)$ against a position-aware threshold $t_{\text{open}} = 6 + 60(\theta_{\text{eff}} - 0.05) - 1.2 \cdot \textit{posBonus} - \textit{stealDrop}$ when unopened, or against an escalating raise threshold $t_{\text{call}} = 6 + 60(\theta_{\text{eff}} - 0.05) + 1.8 \cdot \texttt{raises} + 5 \cdot \texttt{potOdds} + 2.2\max(0, \frac{\texttt{toCall}}{\texttt{bb}} - 3)$ when facing a raise, sizing open-raises to $(2.5 + 0.5\alpha_{\text{eff}}) \cdot \texttt{bb}$ plus 1 bb per limper and 3-bets to $\approx 3.0\times$ the current bet.

On the flop, turn, and river, the postflop decision process (POSTFLOPDECISION, Algorithm 18) combines four game-theoretic safeguards to prevent common AI pitfalls (such as calling down with medium pairs against triple-barrel bets or over-bluffing multi-way pots):

1. **Action-Conditioned Range Floor (c_{floor}):** In heads-up postflop pots ($m = 1$), the engine computes an opponent Chen floor c_{floor} from the preflop raise count (2.5 unraised, 5.5 single-raised, 8.5 in 3-bet+ pots), Bayesian opponent VPIP/PFR reads (± 1.2), multi-raise street aggression ($+0.8$), and facing a bet ($+0.6$):

$$c_{\text{floor}} = \text{clamp}_{[2,10]}\Big(c_{\text{base}}(\texttt{preflopRaises}) + \Delta_{\text{read}} + 0.8 \cdot \mathbf{1}[\texttt{aggressor} = \text{opp} \wedge \texttt{raises} \geq 2] + 0.6 \cdot \mathbf{1}[\texttt{toCall} > 0]\Big), \tag{28}$$

   and evaluates conditional equity $e = \text{EQUITYVSRANGE}(p.H, B, c_{\text{floor}}, N_{\text{iters}})$.

2. **Square-Root Bet-Pressure Discount ($\hat{e}$):** Larger bets polarize and strengthen an opponent's continuing range. Given $\textit{pressure} = \texttt{toCall}/\max(\texttt{bb}, \texttt{pot})$, raw equity e is discounted by a concave square-root penalty and a multi-way penalty:

$$\hat{e} = e - \min\big(c_{\max},\ \gamma_{\text{press}}\sqrt{\textit{pressure}}\big) - 0.025(m - 1), \tag{29}$$

   where $(\gamma_{\text{press}}, c_{\max}) = (0.15, 0.20)$ when e was computed against a uniform range, or $(0.06, 0.10)$ when e was already conditioned on a narrowed range via EQUITYVSRANGE (preventing double-discounting of the opponent's range strength).

3. **Multi-Way Bluff Suppression:** Because the probability that m opponents all fold simultaneously scales as f^m, the effective bluff multiplier β_{eff} is divided by 3.3 whenever $m \geq 2$.

4. **Chip-Step Bet Legalisation (CLAMPRAISE):** All proposed bet and raise sizes (33%, 50%, 66%, 75%, 100%, or 125% of the pot) are rounded to natural casino chip steps (1, 5, or 25) and clamped into $[\tau_{\min}, \tau_{\max}]$.

Algorithm 18: POSTFLOPDECISION — action-conditioned range equity, pressure discounting, and postflop action selection

Input: player p; street state $(\texttt{toCall}, \texttt{pot}, b_{\text{behind}}, \texttt{canRaise}, \texttt{potOdds}, m, \texttt{reads})$
Output: legal postflop action $(a, \tau^\star)$

if $m = 1$ **then** compute c_{floor} via Eq. (28) and $e \leftarrow \textsc{EquityVsRange}(p.H, B, c_{\text{floor}}, N_{\text{iters}})$
if $e =$ **nil then** $(e, _) \leftarrow \textsc{Equity}(p.H, B, m, N_{\text{iters}})$
if $\texttt{toCall} = 0$ **then** ▷ *checked to player p: choose between value bet, thin bet, bluff, or check*
  $\pi_{\text{bet}} \leftarrow 0.86$ if $e > 0.74$, $\min(0.92, 1.15\alpha_{\text{eff}} v_b)$ if $e > 0.62$, $\min(0.90, 0.42\alpha_{\text{eff}} v_b)$ if $e > 0.50$, else $\min(0.85, \beta_{\text{eff}} m_b)$
  if $\texttt{canRaise} \wedge U < \pi_{\text{bet}}$ **then return** $(\textsc{Raise}, \textsc{ClampRaise}(p, \textsc{SizeBet}(e)))$ **else return** CHECK
$\mathit{pressure} \leftarrow \texttt{toCall}/\max(\texttt{bb}, \texttt{pot})$
$\hat{e} \leftarrow e - \min\big(c_{\max}, \gamma_{\text{press}}\sqrt{\mathit{pressure}}\big) - 0.025(m-1)$ ▷ *range & multi-way discount*
$\mathit{need} \leftarrow \texttt{potOdds} + \theta_{\text{eff}} + 0.014\, b_{\text{behind}} + 0.07 \cdot \mathbf{1}[\texttt{raises} \geq 3] + 0.05 \cdot \mathbf{1}[\texttt{toCall} \geq p.\texttt{stack}] - \Delta_{\text{callDown}} - \Delta_{\text{learned}}$
if $e > 0.72 \wedge \texttt{canRaise} \wedge U < 0.92\alpha_{\text{eff}}$ **then return** $(\textsc{Raise}, \textsc{ClampRaise}(p, \textsc{SizeBet}(e)))$ ▷ *value raise*
if $\hat{e} \geq \mathit{need}$ **then return** CALL
if $\texttt{canRaise} \wedge \texttt{raises} < 2 \wedge e > 0.28 \wedge U < 0.4\beta_{\text{eff}} m_b$ **then return** $(\textsc{Raise}, \textsc{ClampRaise}(p, \textsc{SizeBet}(0.6)))$ ▷ *semi-bluff*
if $\texttt{toCall} \leq \texttt{bb} \wedge \hat{e} \geq \mathit{need} - 0.08$ **then return** CALL
return FOLD

8.5 Processes in Associated Full-Game GTO Solvers (CFR, Abstraction, and Subgame Solving)

How do large-scale offline and cloud poker solvers (such as `Slumbot`, `Libratus`, `Pluribus`, `DecisionHoldem`, and commercial GTO solver suites) compute approximate Nash equilibria across the entire multi-street game tree of No-Limit Hold'em, which contains over 10^{160} decision nodes? They execute a three-stage **Counterfactual Regret Minimization (CFR) Process**:

1. **Stage 1 — Information and Action Abstraction:** To shrink the 10^{160}-node tree to a tractable 10^7–10^9 information sets:
   - *Card Bucketing (K-Means over Earth Mover's Distance):* Hands on the flop, turn, and river are clustered into $K \in [200, 5000]$ buckets based on their equity distribution (or potential-aware histogram) across future board runouts.
   - *Action Discretization:* Continuous No-Limit bet sizes are restricted to a sparse grid of pot fractions (e.g., $\{0.33\times, 0.50\times, 0.75\times, 1.0\times, 1.5\times, \text{All-In}\}$), with off-tree human bet sizes mapped to nearby grid points via randomized pseudo-harmonic translation.
2. **Stage 2 — Counterfactual Regret Minimization (CFR) and Regret Matching:** An **information set** I groups all game-tree histories that are indistinguishable to the acting player i (same private cards and same public betting/board sequence). Let $\sigma^t(I, a)$ denote the probability assigned to action $a \in A(I)$ on iteration t, and let $v_i(\sigma^t, I)$ denote player i's expected counterfactual utility at I (assuming player i plays to reach I with probability 1, weighted by the opponent's and chance's reach probability $\pi_{-i}^{\sigma^t}(I)$). The **instantaneous counterfactual regret** of not taking action a at I on iteration t is:

$$r^t(I, a) = v_i\big(\sigma^t_{I \to a}, I\big) - v_i\big(\sigma^t, I\big), \qquad R^T(I, a) = \sum_{t=1}^{T} r^t(I, a). \tag{30}$$

At iteration $T+1$, **Regret Matching** updates the next strategy $\sigma^{T+1}(I,a)$ proportionally to the positive cumulative regrets $R_+^T(I,a) = \max(0, R^T(I,a))$:

$$\sigma^{T+1}(I,a) = \begin{cases} \dfrac{R_+^T(I,a)}{\sum_{a' \in A(I)} R_+^T(I,a')} & \text{if } \sum_{a' \in A(I)} R_+^T(I,a') > 0, \\ \dfrac{1}{|A(I)|} & \text{otherwise.} \end{cases} \tag{31}$$

A fundamental theorem by Zinkevich et al. proves that as $T \to \infty$, overall average regret decays as $O(1/\sqrt{T})$, and the **cumulative average strategy profile** $\bar{\sigma}^T(I,a)$:

$$\bar{\sigma}^T(I,a) = \frac{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)\, \sigma^t(I,a)}{\sum_{t=1}^{T} \pi_i^{\sigma^t}(I)} \tag{32}$$

converges to an exact Nash equilibrium in any two-player zero-sum game. Modern solvers use **Discounted / Linear CFR (CFR$^+$)** (flooring negative regrets to 0 and weighting iteration t by t) or **External-Sampling Monte Carlo CFR (MCCFR)** to accelerate convergence by an order of magnitude.

3. **Stage 3 — Real-Time Depth-Limited Subgame Solving:** When a human makes an off-tree bet postflop, championship engines reconstruct the subgame rooted at the current public state using both players' Bayesian range distributions, run several hundred CFR iterations in real time over the exact unabstracted turn/river subgame, and use neural or rollout leaf value networks at depth limits.

Engine / Solver	Primary Process	Memory / Speed	Best-Suited Application Domain
Reference Engine	Nash Push/Fold + Range MC + Bayesian Reads	$<$ 1 MB, $<$ 25 ms	Instant zero-dependency single-file play with adaptive human exploitation.
Rules Engines (`PokerKit`)	Pure multi-variant state validation	$<$ 5 MB, $<$ 1 ms	Backend hand-history parsing, multi-game rule enforcement, RL environments.
Blueprint CFR (`Slumbot`)	Offline MCCFR over card/bet abstractions	10–100 GB RAM	Fast cloud lookup of full-tree approximate GTO strategies.
Subgame Solvers (`Pluribus`)	Blueprint + real-time depth-limited CFR	Multi-core CPU cluster	Superhuman unexploitable play in deep-stacked cash and 6-max games.
Deep RL (`AlphaHoldem`)	Trinal-Clip PPO end-to-end neural policy	GPU inference (4 ms)	High-speed neural evaluation without explicit abstraction tables.

Table 6: Architectural comparison across poker rules engines, real-time heuristic/Monte Carlo engines, and full-game GTO solvers.

9 Process VII: Bayesian Opponent Profiling and Hindsight-Free Self-Review

9.1 The Beta-Binomial Updating Process and Two-Sigma Gate

A purely static equilibrium strategy never takes full advantage of a human opponent who folds 80% of the time to postflop bets or calls down with any pair. Conversely, a naive adaptive engine that computes raw percentages after just 3 hands will wildly over-adjust to small-sample noise.

The Bayesian opponent-profiling process (BAYESIANREAD, Algorithm 19) models each binary human tendency (`vpip`, `pfr`, `foldToRaise`, `foldToBet`, and `aggr`) as a **Beta-Binomial conjugate prior** $\mathrm{Beta}(a_0, b_0)$ with prior mean $\mu_0 = \frac{a_0}{a_0+b_0}$ (for example, `vpip` $\sim \mathrm{Beta}(5,7)$ with $\mu_0 \approx 41.7\%$, and `foldToBet` $\sim \mathrm{Beta}(6,6)$ with $\mu_0 = 50.0\%$). After observing k occurrences out of n_{obs} opportunities, the exact posterior distribution is $\mathrm{Beta}(a, b)$ with $a = a_0 + k$ and $b = b_0 + (n_{\text{obs}} - k)$, whose posterior mean μ and posterior standard deviation σ are:

$$\mu = \frac{a}{a+b}, \qquad \sigma = \sqrt{\frac{a\,b}{(a+b)^2\,(a+b+1)}}. \tag{33}$$

To prevent small-sample overfitting, the engine outputs a non-zero exploitation signal $z \in [-1,1]$ **if and only if the posterior mean deviates from the prior mean by more than two posterior standard deviations** ($|\mu - \mu_0| > 2\sigma$, roughly a 95% credible interval check):

$$z = \begin{cases} \mathrm{clamp}_{[-1,1]}\left(\dfrac{\mu - \mu_0}{4\sigma}\right) & \text{if } |\mu - \mu_0| > 2\sigma, \\ 0 & \text{otherwise.} \end{cases} \tag{34}$$

Algorithm 19: BAYESIANREAD — Beta-Binomial posterior update, 2σ significance gate, and context-gated exploitative modifiers

Input: prior (a_0, b_0); current posterior (a, b); interaction context (whether target opponent faces the action)

Output: significance-gated signal $z \in [-1,1]$ and exploitative multipliers $(m_b, v_b, \Delta_{\text{callDown}}, \mathit{stealDrop})$

$\mu_0 \leftarrow a_0/(a_0+b_0)$; $\quad n \leftarrow a+b$; $\quad \mu \leftarrow a/n$

$\sigma \leftarrow \sqrt{a\,b/\left(n^2(n+1)\right)}$

$z \leftarrow \mathrm{clamp}_{[-1,1]}\big((\mu-\mu_0)/(4\sigma)\big)$ **if** $|\mu-\mu_0| > 2\sigma$ **else** 0 ▷ 2σ *credible-interval gate*

$m_b \leftarrow \max(0.05, 1 + 1.6\,z_{\text{foldToBet}})$ and $v_b \leftarrow 1 - 0.30\,z_{\text{foldToBet}}$ **if** target opponent can face a bet **else** $(1,1)$

$\Delta_{\text{callDown}} \leftarrow 0.06\max(0, z_{\text{aggr}})$ **if** target opponent is current street aggressor **else** 0

$\mathit{stealDrop} \leftarrow 1.6\max(0, z_{\text{foldToRaise}})$ **if** target opponent has not acted preflop **else** 0

return $(z, m_b, v_b, \Delta_{\text{callDown}}, \mathit{stealDrop})$

9.2 The Hindsight-Free Post-Showdown Self-Review Process (REVIEWHAND)

In addition to profiling opponents, how can an engine calibrate its own calling thresholds across a session without cheating? Two classic design mistakes corrupt naive self-learning poker bots:

1. **Outcome Bias (Result-Oriented Learning):** Judging a call by whether the final river card happened to win or lose the pot teaches the engine to fold 80% favourites after a single bad beat.
2. **Information Leakage:** Inspecting the hole cards of opponents who folded (mucked) gives the AI illicit information that a real player at the table never sees.

The post-showdown self-review process (REVIEWHAND, Algorithm 20) avoids both errors: it records every decision facing a bet (`toCall` > 0) into a trace buffer during the hand, and at the end of the hand it evaluates the **expected value at the moment of the decision** using *only* the hole cards of players who actually revealed their cards at showdown ($q_{\text{true}} =$ EQUITYVSKNOWN),

treating any folded or mucked opponents as unknown random hands. The resulting counterfactual regret (in big blinds) updates a coarse bucket key $(\texttt{stage}, \text{equityTier}, \text{oddsTier})$ capped at ± 0.06 threshold shift.

Algorithm 20: REVIEWHAND — hindsight-free post-showdown expected-value regret update without peeking at mucked cards

Input: set of players *shown* whose cards were revealed at showdown; decision trace buffer $\mathcal{T}$
Output: updated regret buckets $\mathcal{L}[k] = (r_{\text{fold}}, r_{\text{call}}, n)$

$S_{\text{id}} \leftarrow \{q.\texttt{id} : q \in \mathit{shown}\}$
foreach decision record $d = (\mathit{hole}, \mathit{board}, \mathit{oppIds}, \texttt{pot}, \texttt{toCall}, \mathit{act}, k) \in \mathcal{T}$ **do**
  $H_{\text{known}} \leftarrow \{p_i.H : i \in \mathit{oppIds} \cap S_{\text{id}}\}$; $u_{\text{mucked}} \leftarrow |\mathit{oppIds} \setminus S_{\text{id}}|$ ▷ *mucked cards stay hidden*
  $q_{\text{true}} \leftarrow \text{EQUITYVSKNOWN}(\mathit{hole}, H_{\text{known}}, \mathit{board}, u_{\text{mucked}})$
  $\text{EV}_{\text{fold}} \leftarrow 0$; $\text{EV}_{\text{call}} \leftarrow q_{\text{true}} \cdot \texttt{pot} - (1 - q_{\text{true}}) \cdot \texttt{toCall}$
  $\mathit{base} \leftarrow (\text{EV}_{\text{fold}} \text{ if } \mathit{act} = \text{FOLD else } \text{EV}_{\text{call}})$
  $\mathcal{L}[k].r_{\text{fold}} \mathrel{+}= (\text{EV}_{\text{fold}} - \mathit{base})/\texttt{bb}$; $\mathcal{L}[k].r_{\text{call}} \mathrel{+}= (\text{EV}_{\text{call}} - \mathit{base})/\texttt{bb}$; $\mathcal{L}[k].n \mathrel{+}= 1$
Clear trace buffer $\mathcal{T} \leftarrow [\,]$

10 System Invariants, Process Complexity, and Worked Walkthrough

10.1 The Seven Formal System Invariants

Across every deal, action, side-pot split, and showdown, a correct poker engine must satisfy seven mathematical invariants at all times:

The Seven Formal Engine Invariants.

1. **Chip Conservation (Invariant I1):** Let $M_0 = \sum_p p.\texttt{stack}_0$ be total table chips at the start of a hand. At every instant during and after the hand, $\sum_p p.\texttt{stack} + \texttt{pot} = M_0$, and during active play $\texttt{pot} = \sum_p p.\texttt{committed}$.
2. **Non-Negative Stack and Commitment Bounds (Invariant I2):** For every player p, $p.\texttt{stack} \geq 0$, $p.\texttt{bet} \geq 0$, and $p.\texttt{committed} \geq p.\texttt{bet}$. Moreover, $p.\texttt{allIn} = \textbf{true} \iff (\neg p.\texttt{folded} \wedge p.\texttt{stack} = 0 \wedge p.\texttt{committed} > 0)$.
3. **Deck Card Bijection (Invariant I3):** At every street, the multiset union of remaining stub cards, burned cards, board cards B, and all dealt player hole cards $p.H$ has cardinality 52 and equals $\{0, 1, \ldots, 51\}$ without duplicates.
4. **Evaluator Monotonicity and Equivalence (Invariant I4):** For every 7-card subset $A \subset \{0, \ldots, 51\}$, $\textsc{Rank7Fast}(A) = \textsc{Best7}(A) = \max_{C \subset A, |C|=5} \textsc{Eval5}(C)$, and $\textsc{Decode}(\textsc{Encode}(\kappa, k)) = (\kappa, k)$.
5. **Turn-Order Termination (Invariant I5):** Every betting round terminates in a finite number of actions ($\leq N_p(1 + N_{\text{raises}})$), because every action either increments the count of players with $\texttt{hasActed} = \textbf{true}$, folds a player, puts a player all-in, or strictly increases $\texttt{currentBet}$ toward the finite table chip supply M_0.
6. **Side-Pot Eligibility Soundness (Invariant I6):** No player p can ever be eligible for any horizontal commitment band $[L_{j-1}, L_j]$ where $L_j > p.\texttt{committed}$, nor can any folded player belong to any eligibility set E_j.
7. **Statistical Gating and Hindsight Isolation (Invariant I7):** All Bayesian opponent adjustments equal 0 whenever $|\mu - \mu_0| \leq 2\sigma$, and post-hand review never accesses the hole cards of folded or mucked players.

10.2 Summary of Process Complexity and Runtime Benchmarks

Table 7 summarises the asymptotic time complexity, space footprint, and measured single-thread execution latency of each core algorithm in the engine.

10.3 Complete End-to-End Numerical Walkthrough of a Hand Across All Seven Processes

To see how all seven processes work together seamlessly during a single hand, we trace a 3-player table (Seat 0: TAG, Seat 1: Human, Seat 2: LAG) with blinds $\texttt{sb/bb} = 10/20$ and stacks $(1000, 1000, 240)$:

1. **Process I & IV (Blind Advancement & Dealing):** Suppose Seat 0 posted the big blind on the prior hand. STARTHAND advances $\texttt{bbSeat}$ clockwise to Seat 1 (20 chips), sets $\texttt{sbSeat}$ to Seat 0 (10 chips), and sets $\texttt{btnSeat}$ to Seat 2. CRYPTOGRAPHICSHUFFLE permutes the 52-card deck and deals two cards to each player: Seat 2 (BTN) receives A♡ K♢ ($\{49, 46\}$), Seat 0 (SB) receives J♠ J♡ ($\{36, 37\}$), and Seat 1 (BB) receives Q♠ T♠ ($\{40, 32\}$). Total pot is $\texttt{pot} = 30$, $\texttt{currentBet} = 20$, $\texttt{lastRaise} = 20$.
2. **Process VI (Preflop Valuation & Action):** Seat 2 acts first on the button. $\textsc{ChenScore}(\text{A♡}, \text{K♢})$ assigns 10 points for the Ace, -1 for the 1-card gap ($12 - 11 - 1 = 0 \implies 0$ penalty), giving $s = 10.0 \geq t_{\text{open}}$. Seat 2 raises to 55. Seat 0 (J♠ J♡, $\textsc{ChenScore} = 2 \times 6 = 12.0$) calls 45 more (to 55), and Seat 1 (Q♠ T♠, $\textsc{ChenScore} = 7 + 2 - 1 + 1 = 9.0$) calls 35 more (to 55). All

Process / Algorithm	Time Complexity	Space	Measured Latency	Verification & Operational Role
CRYPTOGRAPHICSHUFFLE	$O(52)$	$O(1)$	0.85 ms / hand	Rejection-sampled Fisher–Yates over 52! decks.
ENCODE/DECODE	$O(1)$ (5 steps)	$O(1)$	< 15 ns	Bijective base-13 score packing ($\kappa \cdot 13^5 + \dots$).
BEST7 (EVAL5 $\times 21$)	$O(\binom{7}{5}) = 21$	$O(1)$	41.7 μs / hand	Showdown oracle; identifies exact 5 winning cards.
RANK7FAST	$O(1)$ (1 pass)	48 KB	**564** ns / hand	**1.77**M evals/s; 0 mismatches vs. BEST7 on 200k deals.
EQUITY (N trials)	$O(N \cdot m)$	$O(52)$	1.9 ms ($N = 600$)	Partial Fisher–Yates rollout + fractional pot-share SE.
EQUITYVSRANGE	$O(N_{\text{att}})$	10.8 KB	3.4 ms ($N = 900$)	Rejection sampling over 52×52 Chen matrix.
NEXTCARDSCAN (Turn HU)	$46 \times \binom{45}{2}$	$O(46)$	24.4 ms (exact)	Enumerates all 45,540 deals with SE $= 0$.
COMPUTEPOTS + AWARDPOTS	$O(N_p^2)$	$O(N_p)$	< 0.05 ms	Horizontal commitment slicing & TDA odd-chip rule.
PUSHFOLD	$O(1)$	6.4 KB	< 20 ns	169×19 HU Nash equilibrium lookup (≤ 20 bb).

Table 7: Time complexity, memory footprint, and benchmarked execution latencies across the engine processes.

three players now have `hasActed` = **true** and `bet` = 55, so FINDNEXTACTOR returns -1. The street ends with `pot` = 165.

3. **Process II, III & VI (Flop Evaluation, Range Equity & Short All-In):** One burn card is discarded and the flop comes $B = (\text{Q}\heartsuit, 8\spadesuit, 4\clubsuit)$. Seat 0 checks, and Seat 1 (top pair, Q♠ T♠) bets 100 (`currentBet` $= 100$, `lastRaise` $= 100$). Seat 2 (A♡ K♢, remaining stack $240 - 55 = 185$) moves all-in for 185. Because the raise increment is $\Delta = 185 - 100 = 85 < $ `lastRaise` $= 100$, this is a **short all-in**: `currentBet` becomes 185, `lastRaise` remains 100, and Seat 1's `hasActed` flag remains **true**.

   - Seat 0 (J♠ J♡) had only checked (`actedAtBet` $= 0$), so $185 - 0 = 185 \geq 20$: Seat 0 has full action, runs POSTFLOPDECISION, sees $\hat{e}$ below *need* in a 3-way pot facing a bet and an all-in, and folds (leaving 55 dead chips in the pot).
   - Action returns to Seat 1 (`bet` $= 100 < 185$). Checking CANREOPEN(Seat 1) yields $185 - 100 = 85 <$ `minRaiseSeen` $= 100 \implies$ **false**: Seat 1 may only call the additional 85 or fold. Seat 1 calls 85. The pot is now `pot` $= 55 + 240 + 240 = 535$.

4. **Process III (Turn Exact Enumeration):** Because Seat 2 is all-in ($n_{\text{canAct}} = 1$), the turn 2♢ and river 7♣ are dealt out. On the turn $B_4 = (\text{Q}\heartsuit, 8\spadesuit, 4\clubsuit, 2\diamondsuit)$, NEXTCARDSCAN knows only 44 unseen cards remain between Seat 1 and Seat 2: Seat 2 (A♡ K♢) needs one of the 3 remaining Aces or 3 remaining Kings (6 outs), giving exact river equity $6/44 = 13.64\%$, while Seat 1 (Q♠ T♠) holds 86.36% equity.

5. **Process V (Side-Pot Slicing & Showdown Settlement):** After the river 7♣ falls, commitments are $p_0.\texttt{committed} = 55$ (folded), $p_1.\texttt{committed} = 240$ (live), and $p_2.\texttt{committed} = 240$ (live).
   - REFUNDUNCALLED sees $c_1 = c_2 = 240$, so $\Delta_{\text{uncalled}} = 0$.
   - COMPUTEPOTS slices Band 1 ($0 \to 55$, amount $3 \times 55 = 165$, $E_1 = \{1, 2\}$) and Band 2 ($55 \to 240$, amount $2 \times 185 = 370$, $E_2 = \{1, 2\}$), and merges them because $E_1 = E_2 = \{1, 2\}$ into a single Main Pot of 535.
   - AWARDPOTS scores Seat 1's Pair of Queens ($\kappa = 1, k = (10, 8, 6, 5)$, score $V = 675,519$) above Seat 2's Ace-High ($\kappa = 0, k = (12, 11, 10, 6, 5) \implies V = 368,243$), awarding all 535 chips to Seat 1. Total table chips remain $945 + 1295 + 0 = 2240 = M_0$ (Invariant I1).